

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Thiết kế xây dựng công nghệ thực tế ảo và ứng dụng

TRẦN QUANG HUY

nguyenvanabc@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: PGS. TS. Phạm Văn ABC

Khoa: Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2022

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Thiết kế xây dựng công nghệ thực tế ảo và ứng dụng

TRẦN QUANG HUY
nguyenvanabc@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: PGS. TS. Phạm Văn ABC

Chữ kí GVHD

Khoa:

Kỹ thuật máy tính

Trường:

Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2022

LỜI CẢM ƠN

Lời cảm ơn của sinh viên (SV) tới người yêu, gia đình, bạn bè, thầy cô, và chính bản thân mình vì đã chăm chỉ và quyết tâm thực hiện ĐATN để đạt kết quả tốt nhất, nên viết phần cảm ơn ngắn gọn, tránh dùng các từ sáo rỗng, giới hạn trong khoảng 100-150 từ.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Sinh viên viết tóm tắt ĐATN của mình trong mục này, với 200 đến 350 từ. Theo trình tự, các nội dung tóm tắt cần có: (i) Giới thiệu vấn đề (tại sao có vấn đề đó, hiện tại được giải quyết chưa, có những hướng tiếp cận nào, các hướng này giải quyết như thế nào, hạn chế là gì), (ii) Hướng tiếp cận sinh viên lựa chọn là gì, vì sao chọn hướng đó, (iii) Tổng quan giải pháp của sinh viên theo hướng tiếp cận đã chọn, và (iv) Đóng góp chính của ĐATN là gì, kết quả đạt được sau cùng là gì. Sinh viên cần viết thành đoạn văn, không được viết ý hoặc gạch đầu dòng.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

Mục này khuyến khích sinh viên viết lại mục “Tóm tắt” đồ án tốt nghiệp ở trang trước bằng tiếng Anh. Phần này phải có đầy đủ các nội dung như trong phần tóm tắt bằng tiếng Việt. Sinh viên không nhất thiết phải trình bày mục này.

Nhưng nếu lựa chọn trình bày, sinh viên cần đảm bảo câu từ và ngữ pháp chuẩn tắc, nếu không sẽ có tác dụng ngược, gây phản cảm.

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	1
1.1 Đặt vấn đề.....	1
1.2 Các giải pháp hiện có và hạn chế.....	1
1.3 Mục tiêu và định hướng giải pháp	2
1.4 Đóng góp đề án	2
1.5 Bố cục đề án	3
CHƯƠNG 2. TỔNG QUAN VÀ CÔNG TRÌNH LIÊN QUAN.....	4
2.1 Kiến trúc RAG tổng quan.....	4
2.1.1 Hạn chế của LLM thuần túy.....	4
2.1.2 Kiến trúc RAG.....	4
2.2 Truy xuất thông tin	5
2.2.1 BM25 – Sparse Retrieval	5
2.2.2 Dense Retrieval – Bi-encoder.....	6
2.2.3 Hybrid Retrieval và Reciprocal Rank Fusion	7
2.3 Reranking – Cross-encoder.....	8
2.3.1 Kiến trúc Cross-encoder và chi phí suy luận	8
2.3.2 Knowledge Distillation tổng quan.....	9
2.3.3 Các loss functions cho reranker KD	9
2.4 Mô hình nhúng văn bản.....	11
2.4.1 Kiến trúc Transformer và BERT	11
2.4.2 Vietnamese_Embedding_v2 và BGE-M3	12
2.4.3 MultipleNegativesRankingLoss.....	12
2.4.4 Matryoshka Representation Learning	13
2.4.5 Curriculum Learning	13

2.5 Mô hình ngôn ngữ lớn trong pipeline RAG	14
2.5.1 GPT-4o-mini	14
2.5.2 Chain-of-Thought Prompting	14
2.6 Query Routing và phân loại Intent.....	14
2.6.1 LLM-based Routing.....	14
2.6.2 PhoBERT và phân loại văn bản tiếng Việt	15
2.7 Research Gap	15
CHƯƠNG 3. HỆ THỐNG RAG BAN ĐẦU (V1)	17
3.1 Vai trò của hệ thống v1 trong đồ án	17
3.2 Quy trình tiền xử lý tài liệu.....	17
3.2.1 OCR và chuẩn hóa văn bản	17
3.2.2 TableAwareChunker.....	18
3.2.3 Xây dựng chỉ mục	19
3.3 Kiến trúc ba pipeline RAG	19
3.3.1 Pipeline Baseline	19
3.3.2 Pipeline Fusion	20
3.3.3 Pipeline Router-Summary	21
3.3.4 So sánh kiến trúc ba pipeline.....	21
3.4 Đánh giá thực nghiệm v1.....	22
3.4.1 Thiết lập thực nghiệm.....	22
3.4.2 Kết quả tổng thể và theo intent	22
3.4.3 Phân tích chất lượng retrieval.....	23
3.4.4 Phân tích lỗi định tính	23
3.5 Xác định bottleneck	24
3.5.1 Bottleneck latency: Reranker BGE-M3	24
3.5.2 Bottleneck chi phí: GPT Intent Router	24

3.5.3 Bottleneck chất lượng: Embedding chưa tối ưu domain.....	24
CHƯƠNG 4. CÁC PHƯƠNG PHÁP TỐI ƯU ĐỀ XUẤT (V2)	26
4.1 Tổng quan định hướng v2.....	26
4.2 Lightweight Reranker via Knowledge Distillation	26
4.2.1 Motivation và thiết kế tổng quan	26
4.2.2 Pipeline xây dựng dữ liệu huấn luyện.....	27
4.2.3 Stage A: Contrastive Learning.....	29
4.2.4 Stage B: Knowledge Distillation.....	29
4.3 Fine-tune Embedding bằng MRL và Curriculum Learning	31
4.3.1 Pipeline xây dựng dữ liệu embedding	31
4.3.2 Matryoshka Loss và MNR	32
4.3.3 Curriculum Training ba giai đoạn	33
4.3.4 Chiến lược deployment với MRL	33
4.4 Fine-tune PhoBERT Intent Classifier.....	34
4.4.1 Kiến trúc và dữ liệu.....	34
4.4.2 Xử lý class imbalance.....	34
4.4.3 Routing logic trong pipeline v2	35
CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ	36
5.1 Thiết lập thực nghiệm	36
5.1.1 Môi trường và phần cứng.....	36
5.1.2 Tập dữ liệu đánh giá.....	36
5.1.3 Metrics đánh giá	36
5.2 Đánh giá Reranker.....	37
5.2.1 Ablation study: xác nhận vai trò của Stage A	37
5.2.2 Ablation study: ảnh hưởng của mMARCO-VI.....	38
5.2.3 So sánh các loss functions và teacher models trong Stage B.....	38

5.2.4 So sánh với baselines	40
5.2.5 Phân tích lỗi reranker	41
5.3 Đánh giá Embedding	42
5.3.1 Kết quả theo từng curriculum stage.....	42
5.3.2 Ablation theo dimension với MRL.....	42
5.3.3 So sánh với baseline.....	43
5.4 Đánh giá Intent Classifier	43
5.5 So sánh end-to-end v1 và v2	44
5.6 Phân tích lỗi tổng thể	44
CHƯƠNG 6. KẾT LUẬN.....	46
6.1 Contributions	46
6.2 Limitations.....	46
6.3 Future Works.....	46
TÀI LIỆU THAM KHẢO	49
PHỤ LỤC	51
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP.....	51
A.1 Ngành học.....	52
A.2 Đánh dấu (bullet) và đánh số (numering)	52
A.3 Cách thêm bảng.....	53
A.4 Chèn hình ảnh	53
A.5 Tài liệu tham khảo	54
A.6 Cách viết phương trình và công thức toán học	54
A.7 Qui cách đóng quyển.....	54
B. ĐẶC TẢ USE CASE	56
B.1 Đặc tả use case “Thống kê tình hình mượn sách”	56
B.2 Đặc tả use case “Đăng ký làm thẻ mượn”	56

DANH MỤC HÌNH VẼ

Hình A.1	Internet vạn vật	53
Hình A.2	Qui cách đóng quyển đồ án	55
Hình A.3	Qui cách đóng quyển đồ án	55

DANH MỤC BẢNG BIỂU

Bảng 2.1	Ví dụ kết hợp thứ hạng bằng RRF	7
Bảng 2.2	So sánh bi-encoder và cross-encoder	8
Bảng 2.3	Ví dụ target và student score trong ADR-MSE	11
Bảng 2.4	Ví dụ in-batch negatives với batch size bằng 3	12
Bảng 3.1	Vai trò của các tập dữ liệu trong quy trình phát triển	17
Bảng 3.2	Ví dụ bảng thông số được bảo toàn sau OCR	18
Bảng 3.3	So sánh kiến trúc ba pipeline v1	21
Bảng 3.4	Cấu hình thực nghiệm v1 trên tập phát triển	22
Bảng 3.5	Accuracy tổng thể và theo intent của ba pipeline v1	22
Bảng 3.6	Phân tích nguồn gốc lỗi của Baseline và Router-Summary	23
Bảng 3.7	Phân loại nguyên nhân lỗi trên 27 câu của Router-Summary	23
Bảng 3.8	So sánh latency của các reranker trên CPU	24
Bảng 4.1	Ánh xạ bottleneck của v1 sang giải pháp v2	26
Bảng 4.2	Thống kê synthesized negatives sau lọc	28
Bảng 4.3	Thống kê mined negatives theo nguồn	28
Bảng 4.4	Tổng quan dataset huấn luyện reranker	28
Bảng 4.5	Hyperparameters của Stage A	29
Bảng 4.6	Ví dụ tính ADR-MSE với năm candidates	31
Bảng 4.7	Hyperparameters của Stage B	31
Bảng 4.8	Các nguồn negative trong master dataset embedding	32
Bảng 4.9	Cấu hình curriculum training cho embedding	33
Bảng 4.10	Các chế độ deployment từ một checkpoint MRL	33
Bảng 4.11	Hyperparameters fine-tune PhoBERT	35
Bảng 4.12	So sánh GPT router của v1 và PhoBERT classifier của v2	35
Bảng 5.1	Các tập dữ liệu dùng trong đánh giá	36
Bảng 5.2	Ablation về vai trò của Stage A và hệ số α	37
Bảng 5.3	So sánh các hướng tích hợp mMARCO-VI	38
Bảng 5.4	So sánh các loss functions và teacher models trong Stage B	39
Bảng 5.5	So sánh MiniLM-ADR-MSE với các baselines	40
Bảng 5.6	Đánh giá end-to-end các reranker trên 343 câu hỏi trắc nghiệm	40
Bảng 5.7	Phân tích 31 miss cases của ADR-MSE (BGE-M3 teacher)	41
Bảng 5.8	Tiến trình cải thiện qua 3 curriculum stages	42
Bảng 5.9	Kết quả MRL theo kích thước embedding	42

Bảng 5.10	So sánh embedding fine-tuned với baseline	43
Bảng 5.11	Khung kết quả đánh giá PhoBERT Intent Classifier	43
Bảng 5.12	Khung so sánh end-to-end giữa v1 và v2	44
Bảng A.1	Table to test captions and labels.	53

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
API	Giao diện lập trình ứng dụng (Application Programming Interface)
EUD	Phát triển ứng dụng người dùng cuối(End-User Development)
GWT	Công cụ lập trình Javascript bằng Java của Google (Google Web Toolkit)
HTML	Ngôn ngữ đánh dấu siêu văn bản (HyperText Markup Language)
IaaS	Dịch vụ hạ tầng

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Sự bùng nổ của các mô hình ngôn ngữ lớn (LLM) đã mở ra hướng tiếp cận mới cho bài toán hỏi đáp tự động. Tuy nhiên, LLM thuần túy bộc lộ hạn chế rõ rệt khi cần trả lời câu hỏi về kiến thức chuyên biệt, cập nhật theo thời gian hoặc đòi hỏi tham chiếu tài liệu cụ thể — những tình huống thường xuyên gặp trong môi trường doanh nghiệp. Kiến trúc Retrieval-Augmented Generation (RAG) ra đời nhằm giải quyết vấn đề này bằng cách kết hợp truy xuất thông tin từ kho tài liệu nội bộ với khả năng sinh ngôn ngữ của LLM, giúp câu trả lời bám sát nguồn và có thể kiểm chứng.

Đề án này được thực hiện trên tập dữ liệu của cuộc thi Viettel AI Race 2025, gồm 991 câu hỏi trắc nghiệm tiếng Việt trải rộng trên 49 tài liệu kỹ thuật đa lĩnh vực. Tập tài liệu có cấu trúc phức tạp — bảng biểu, ký hiệu kỹ thuật, thuật ngữ chuyên ngành — và đặt ra thách thức không nhỏ cho các bước chunking, retrieval lẫn ranking. Đây là bối cảnh thực tiễn phản ánh đúng những khó khăn khi triển khai RAG trong môi trường sản xuất thực tế, không phải trên các benchmark học thuật lý tưởng hóa.

1.2 Các giải pháp hiện có và hạn chế

Các hệ thống RAG hiện nay thường kết hợp Hybrid Retrieval (BM25 + Dense + RRF), Cross-Encoder Reranker, và LLM generation. Đây là kiến trúc được chứng minh hiệu quả về mặt học thuật. Tuy nhiên, trong quá trình xây dựng và đánh giá hệ thống trên tập dữ liệu cuộc thi, chúng tôi quan sát thấy ba vấn đề thực tế mà các công trình hiện có chưa giải quyết triệt để:

- Reranker quá nặng cho triển khai thực tế. BGE-M3 (568M tham số) là reranker mạnh nhưng tạo ra bottleneck latency nghiêm trọng — khoảng 147ms mỗi query — khiến toàn bộ pipeline bị nghẽn tại bước này. Các công trình hiện tại thường chỉ báo cáo accuracy mà bỏ qua chi phí suy luận, trong khi đây là yếu tố quyết định khả năng triển khai thực tế.
- Routing bằng LLM lớn tốn kém không cần thiết. Hệ thống Router-Summary sử dụng GPT-4o-mini để phân loại intent — một tác vụ thực chất chỉ cần phân biệt hai lớp (`tra_cuu` / `ting_toan`). Mỗi lần gọi API thêm khoảng 800ms latency và phát sinh chi phí, trong khi accuracy phân loại đạt 99.4% — ngưỡng này hoàn toàn có thể đạt được bằng mô hình nhỏ hơn nhiều bậc.
- Embedding pretrained chưa tối ưu cho domain. Mô hình Vietnamese Embed-

ding v2, dù là embedding mạnh nhất hiện có cho tiếng Việt, vẫn chỉ đạt $\text{Acc}@1 = 73.89\%$ trên tập test nội bộ khi đánh giá riêng bước retrieval. Đây là giới hạn trên của toàn pipeline — nếu retrieval không kéo đúng chunk, các bước sau không thể cứu vãn.

1.3 Mục tiêu và định hướng giải pháp

Đề án hướng tới mục tiêu xây dựng và tối ưu hệ thống RAG tiếng Việt theo hai giai đoạn phát triển có căn cứ thực nghiệm.

Giai đoạn 1 xây dựng và đánh giá ba pipeline RAG — Baseline, Fusion và Router-Summary — trên tập 991 câu hỏi, nhằm có bức tranh định lượng rõ ràng về accuracy, latency và các dạng lỗi thực tế trước khi quyết định tối ưu.

Giai đoạn 2 dựa trên kết quả giai đoạn 1 đề xuất ba hướng tối ưu có chủ đích, mỗi hướng giải quyết đúng một bottleneck đã xác định:

- Fine-tune lightweight reranker MiniLM-L12 (33M tham số) thông qua Knowledge Distillation từ BGE-M3 (568M), nhằm giảm latency reranking khoảng 6–7 lần trong khi duy trì chất lượng ranking.
- Fine-tune PhoBERT phân loại intent thay thế GPT-4o-mini, nhằm loại bỏ chi phí API và giảm latency routing.
- Fine-tune embedding model với Matryoshka Representation Learning và Curriculum Training, nhằm cải thiện chất lượng retrieval trên domain tài liệu kỹ thuật tiếng Việt.

1.4 Đóng góp đề án

Các đóng góp chính của đề án bao gồm:

- **[C1] Bộ benchmark RAG ba pipeline.** Đánh giá toàn diện Baseline, Fusion và Router-Summary trên 991 câu hỏi tiếng Việt, bao gồm phân tích theo intent, chất lượng retrieval và phân loại nguyên nhân lỗi định tính trên 27 câu sai của pipeline tốt nhất.
- **[C2] Pipeline xây dựng dữ liệu năm giai đoạn cho lightweight reranker.** Từ GPT judge xác định positive, tổng hợp hard negatives (synthesized + mined), lọc answerability độc lập, đến merge và split train/dev — tạo ra 2,994 triplets với xác minh chất lượng đa tầng.
- **[C3] ADR-MSE loss function.** Sử dụng rank positions làm teacher signal thay vì continuous scores, tránh scale mismatch giữa cross-encoder logits và retrieval scores. ADR-MSE đạt $\text{MRR}@10 = 0.7827$, vượt PhoRanker 135M (+0.0412) với model nhỏ hơn 4 lần.

- **[C4] Phân tích so sánh các loss function cho reranker KD.** So sánh thực nghiệm sáu loss function — ListwiseKL, Margin-MSE, RankNet, ADR-MSE, Discounted ADR, Top-K ADR — trên cùng dữ liệu và điều kiện, rút ra nhận xét về ảnh hưởng của quy mô dữ liệu và chất lượng teacher signal.
- **[C5] Hệ thống RAG tối ưu end-to-end.** Tích hợp lightweight reranker, PhoBERT intent classifier và fine-tuned embedding giảm latency và chi phí inference đáng kể so với hệ thống v1 trong khi duy trì accuracy tổng thể.

1.5 Bố cục đề án

Nội dung còn lại của đề án được tổ chức như sau:

- Chương 2 trình bày nền tảng lý thuyết của các kỹ thuật được sử dụng, từ BM25, dense retrieval, cross-encoder reranker, đến knowledge distillation, MRL và PhoBERT.
- Chương 3 mô tả hệ thống RAG ban đầu (v1): quy trình tiền xử lý tài liệu, kiến trúc ba pipeline và kết quả đánh giá thực nghiệm, bao gồm xác định các bottleneck dẫn đến giai đoạn tối ưu.
- Chương 4 trình bày ba phương pháp tối ưu đề xuất — lightweight reranker, fine-tune embedding và fine-tune intent classifier — cùng thiết kế dữ liệu huấn luyện cho từng phương pháp.
- Chương 5 báo cáo kết quả thực nghiệm toàn diện, bao gồm ablation study, so sánh với các baseline và đánh giá end-to-end v1 so với v2.
- Chương 6 đưa ra kết luận, thảo luận hạn chế và định hướng phát triển.

CHƯƠNG 2. TỔNG QUAN VÀ CÔNG TRÌNH LIÊN QUAN

2.1 Kiến trúc RAG tổng quan

2.1.1 Hạn chế của LLM thuần túy

Các mô hình ngôn ngữ lớn (Large Language Models – LLM) như GPT-4, Llama hay Qwen đã đạt được thành tựu nổi bật trong nhiều tác vụ xử lý ngôn ngữ tự nhiên. Tuy nhiên, khi triển khai trong môi trường doanh nghiệp với kho tài liệu nội bộ, LLM thuần túy bộc lộ ba hạn chế căn bản.

Hallucination.

Đây là hiện tượng LLM tạo ra thông tin nghe hợp lý nhưng không chính xác. Do LLM là mô hình xác suất sinh token tiếp theo dựa trên các mẫu đã học, nó không có cơ chế tự xác minh thông tin với nguồn bên ngoài. Khi câu hỏi nằm ngoài tri thức huấn luyện, model có thể tạo ra câu trả lời thay vì thừa nhận không biết. Hậu quả đặc biệt nghiêm trọng trong môi trường kỹ thuật, nơi câu trả lời cần độ chính xác cao.

Knowledge cutoff.

Đây là giới hạn thời gian của tri thức: model không biết thông tin xuất hiện sau thời điểm huấn luyện và không thể cập nhật kiến thức nếu không fine-tune lại, vốn tốn kém tài nguyên tính toán.

Domain gap.

Đây là khoảng cách giữa corpus huấn luyện chung, chẳng hạn dữ liệu web, Wikipedia và sách, với tài liệu kỹ thuật chuyên ngành nội bộ. LLM không được huấn luyện trên tài liệu đặc thù của từng tổ chức nên có thể trả lời thiếu chính xác về thông số, quy trình hoặc chính sách riêng.

2.1.2 Kiến trúc RAG

Retrieval-Augmented Generation (Lewis et al., 2020) giải quyết các hạn chế trên bằng cách bổ sung bước truy xuất thông tin trước khi sinh câu trả lời. Thay vì chỉ dựa vào tri thức nằm trong tham số model, RAG cung cấp cho LLM context cụ thể từ kho tài liệu, qua đó neo câu trả lời vào nguồn có thể kiểm chứng.

Quy trình RAG gồm hai giai đoạn chính. Trong **giai đoạn offline** (indexing), tài liệu được chia thành các đoạn nhỏ, mỗi đoạn được mã hóa thành vector embedding và lưu vào vector database. Trong **giai đoạn online** (inference), hệ thống mã hóa query đầu vào, tìm các chunk tương đồng nhất, ghép context vào prompt và yêu cầu LLM sinh câu trả lời dựa trên context được cung cấp.

Chất lượng hệ thống RAG phụ thuộc chủ yếu vào hai thành phần: **retrieval**, chịu trách nhiệm đưa đúng chunk vào candidate pool, và **reranking**, chịu trách nhiệm sắp xếp các chunk theo mức độ liên quan. Nếu chunk chứa thông tin cần thiết không xuất hiện trong top-k, bước generation khó có thể tạo ra câu trả lời đúng dù LLM mạnh. Đây là lý do đề án tập trung tối ưu retrieval và reranking thay vì thay thế LLM.

Trong bài toán hỏi đáp tài liệu kỹ thuật tiếng Việt, 991 câu hỏi ban đầu trên 200 tài liệu PDF được sử dụng để chạy thử pipeline v1, phân tích bottleneck và xây dựng dữ liệu huấn luyện cho v2. RAG phù hợp với bài toán này vì kho tài liệu có thể được cập nhật độc lập với model, câu trả lời cần tham chiếu nguồn cụ thể, và nhiều câu hỏi đòi hỏi thông tin chi tiết như số liệu, thông số kỹ thuật hoặc điều kiện vận hành mà LLM khó ghi nhớ chính xác.

2.2 Truy xuất thông tin

2.2.1 BM25 – Sparse Retrieval

BM25 (Best Matching 25) là mô hình truy xuất xác suất được Robertson và Zaragoza đề xuất năm 2009, phát triển từ mô hình xác suất cơ bản của Robertson và Sparck Jones (1976). BM25 vẫn là baseline quan trọng trong nhiều hệ thống truy xuất hiện đại vì hiệu quả, không yêu cầu GPU và đặc biệt mạnh với từ khóa kỹ thuật chính xác.

Điểm số BM25 của tài liệu d đối với query q gồm các term q_i được tính như sau:

$$\text{BM25}(d, q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, d)(k_1 + 1)}{f(q_i, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}$$

trong đó $f(q_i, d)$ là tần suất xuất hiện của term q_i trong tài liệu d , $|d|$ là độ dài tài liệu và avgdl là độ dài trung bình của toàn corpus. Siêu tham số $k_1 \in [1.2, 2.0]$ kiểm soát mức bão hòa của term frequency; giá trị lớn hơn cho phép tần suất tiếp tục làm tăng điểm khi term xuất hiện nhiều lần. Siêu tham số $b = 0.75$ kiểm soát chuẩn hóa độ dài; $b = 0$ tương ứng không chuẩn hóa và $b = 1$ tương ứng chuẩn hóa hoàn toàn theo avgdl.

IDF được tính theo công thức:

$$\text{IDF}(q_i) = \log \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}$$

trong đó N là tổng số tài liệu và $n(q_i)$ là số tài liệu chứa term q_i . IDF giảm trọng

số của các từ phổ biến và tăng trọng số của các từ hiếm. Tính chất này đặc biệt hữu ích với thuật ngữ kỹ thuật chuyên ngành xuất hiện ít trong corpus.

Ví dụ minh họa.

Xét một corpus gồm ba tài liệu và query “nhiệt độ vận hành tối đa”:

- Doc 1: “nhiệt độ vận hành tối đa là 85°C”, chứa đầy đủ các term của query.
- Doc 2: “nhiệt độ môi trường”, chỉ chứa một phần term.
- Doc 3: “áp suất vận hành”, chỉ chứa một phần term.

BM25 xếp Doc 1 cao nhất vì chứa nhiều term của query nhất, đồng thời vẫn điều chỉnh điểm nếu tài liệu dài bất thường so với avgdl. Term “nhiệt độ” xuất hiện trong nhiều tài liệu nên có IDF thấp hơn term “tối đa”, vốn chỉ xuất hiện trong Doc 1.

Ưu điểm thực tiễn của BM25 gồm ba điểm: không cần GPU và có thể lập chỉ mục nhanh; hiệu quả với exact keyword match, đặc biệt với mã tài liệu như `Publ ic_021`, tên model thiết bị hoặc ký hiệu kỹ thuật; và ít chịu ảnh hưởng của domain shift vì chỉ dựa trên tần suất từ. Hạn chế chính là không nắm bắt đầy đủ quan hệ ngữ nghĩa. Chẳng hạn, query dùng cụm “nhiệt độ hoạt động” có thể không khớp tốt với tài liệu chỉ sử dụng thuật ngữ tiếng Anh tương đương.

2.2.2 Dense Retrieval – Bi-encoder

Dense retrieval sử dụng kiến trúc bi-encoder để mã hóa query và document thành các vector dày đặc trong cùng một không gian ngữ nghĩa. Khác với BM25 chỉ dựa vào mức độ trùng khớp từ, bi-encoder học biểu diễn ngữ nghĩa nên hai câu khác nhau về mặt chữ nhưng gần nghĩa có thể có vector gần nhau.

Kiến trúc bi-encoder gồm hai encoder xử lý query và document độc lập:

$$s(q, d) = \cos(\mathbf{E}_q(q), \mathbf{E}_d(d)) = \frac{\mathbf{E}_q(q) \cdot \mathbf{E}_d(d)}{\|\mathbf{E}_q(q)\| \cdot \|\mathbf{E}_d(d)\|}$$

trong đó $\mathbf{E}_q$ và $\mathbf{E}_d$ lần lượt là encoder của query và document. Trong thực tế, nhiều hệ thống sử dụng cùng một encoder với shared weights cho cả hai đầu vào.

Toàn bộ document embeddings được tính trước ở giai đoạn offline và lưu trong vector database. Khi inference, hệ thống chỉ cần encode query một lần rồi thực hiện approximate nearest neighbor search. Cơ chế này cho latency thấp ngay cả khi corpus lớn và là lợi thế quan trọng so với cross-encoder reranker, vốn không thể precompute.

Ví dụ minh họa.

Với query “thiết bị có thể chịu nhiệt độ bao nhiêu?”, dense retrieval có thể tìm được chunk “nhiệt độ hoạt động tối đa: 85°C” dù hai câu chỉ trùng khớp ít từ, vì chúng có biểu diễn ngữ nghĩa gần nhau. BM25 có thể xếp trường hợp này thấp hơn nếu tài liệu và query sử dụng các cách diễn đạt khác nhau.

Hạn chế chính của dense retrieval là chất lượng phụ thuộc vào embedding model. Nếu model chưa được tối ưu cho domain, vector của query và document liên quan có thể nằm xa nhau. Đây là một bottleneck được phát hiện trong hệ thống v1, dẫn đến việc fine-tune embedding ở v2.

2.2.3 Hybrid Retrieval và Reciprocal Rank Fusion

Sparse và dense retrieval có điểm mạnh bổ sung cho nhau: BM25 bắt được exact keyword match, trong khi dense retrieval bắt được quan hệ ngữ nghĩa. Hybrid retrieval kết hợp cả hai để tăng recall tổng thể, tức tăng xác suất gold chunk xuất hiện trong candidate pool trước khi reranking.

Thách thức khi kết hợp là hai retriever trả về điểm số trên các thang đo khác nhau. Điểm BM25 là số thực dương không bị chặn, còn cosine similarity thường nằm trong $[-1, 1]$. Việc cộng trực tiếp các điểm số có thể làm kết quả bị chi phối bởi scale của một retriever.

Reciprocal Rank Fusion – RRF (Cormack et al., 2009) giải quyết vấn đề này bằng cách chỉ sử dụng thứ hạng:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

trong đó R là tập retriever, $\text{rank}_r(d)$ là thứ hạng của document d trong retriever r , với rank 1 là tốt nhất, và $k = 60$ là hằng số ổn định thường được sử dụng.

Table 2.1: Ví dụ kết hợp thứ hạng bằng RRF

Document	Rank BM25	Rank Dense	RRF ($k = 60$)
Doc A	1	3	$1/61 + 1/63 = 0.0321$
Doc B	2	1	$1/62 + 1/61 = 0.0326$
Doc C	10	2	$1/70 + 1/62 = 0.0304$

Doc B được xếp cao nhất vì xuất hiện trong top-2 của cả hai retriever, dù không phải lúc nào cũng đứng đầu. RRF thưởng cho sự nhất quán giữa các retriever. Hằng số $k = 60$ làm phẳng chênh lệch giữa các vị trí thấp, nhờ đó hạn chế một outlier từ một retriever chi phối kết quả.

Trong hệ thống đồ án, BM25 top-10 và ChromaDB dense top-10 được hợp nhất

bằng RRF với $k = 60$ để tạo candidate pool trước khi đưa vào reranker.

2.3 Reranking – Cross-encoder

2.3.1 Kiến trúc Cross-encoder và chi phí suy luận

Cross-encoder là kiến trúc reranker nhận đồng thời cặp query và document làm đầu vào, cho phép cross-attention đầy đủ giữa từng token của hai chuỗi. Khác với bi-encoder mã hóa hai đầu vào độc lập, cross-encoder xử lý chúng như một chuỗi ghép nối:

$$\text{Input} = [\text{CLS}] \ q_1 \ q_2 \ \dots \ q_m \ [\text{SEP}] \ d_1 \ d_2 \ \dots \ d_n \ [\text{SEP}]$$

$$s(q, d) = \text{Linear}(\mathbf{h}_{[\text{CLS}]})$$

trong đó $\mathbf{h}_{[\text{CLS}]}$ là hidden state của token $[\text{CLS}]$ sau Transformer encoder, còn lớp Linear tạo ra một scalar relevance score.

Cross-attention đầy đủ cho phép model nắm bắt quan hệ token–token giữa query và document. Ví dụ, từ “tối đa” trong query có thể attend trực tiếp đến giá trị “85°C” trong document. Đây là lý do cross-encoder thường cho chất lượng ranking cao hơn bi-encoder.

Hạn chế chính là document embeddings không thể được tính trước. Với mỗi query mới, tất cả cặp query–candidate phải được xử lý lại. Latency vì vậy tăng gần tuyến tính theo số candidates; rerank 20 candidates yêu cầu chấm điểm 20 cặp đầu vào.

Table 2.2: So sánh bi-encoder và cross-encoder

Đặc điểm	Bi-encoder	Cross-encoder
Precompute document	Có, thực hiện offline	Không
Latency	$O(1)$ lần encode query	$O(n)$ candidates
Chất lượng ranking	Trung bình	Cao
Vai trò trong pipeline	First-stage retrieval	Second-stage reranking

Trong pipeline thực tế, bi-encoder truy xuất nhanh khoảng 20–50 candidates, sau đó cross-encoder rerank để chọn top-5 cuối cùng. Cả v1 và v2 của đồ án đều sử dụng thiết kế này. Với BAAI/bge-reranker-v2-m3 gồm 568 triệu tham số và 20 candidates, latency reranking trên CPU đạt khoảng 147ms/query. Đây là bottleneck chính được xác định trong v1.

2.3.2 Knowledge Distillation tổng quan

Knowledge Distillation – KD (Hinton et al., 2015) là kỹ thuật chuyển tri thức từ một teacher model lớn sang student model nhỏ hơn. Thay vì chỉ học từ hard labels, student học từ soft outputs của teacher như phân phối xác suất hoặc điểm số liên tục.

Soft labels chứa nhiều thông tin hơn hard labels. Chẳng hạn, với câu hỏi về nhiệt độ vận hành, một chunk có thể chứa nhiệt độ tối đa và một chunk khác chứa dải nhiệt độ khuyến nghị. Hard labels chỉ đánh dấu gold chunk là 1 và chunk còn lại là 0, trong khi điểm teacher có thể biểu diễn rằng chunk thứ hai vẫn liên quan một phần.

Cơ chế KD cơ bản được mô tả bởi:

$$\mathcal{L}_{KD} = \text{CE}(\text{softmax}(z^S/T), \text{softmax}(z^T/T))$$

trong đó z^S và z^T là logits của student và teacher, còn T là temperature. Khi $T > 1$, phân phối softmax mềm hơn, giúp student nhận được tín hiệu về quan hệ giữa các đầu ra thay vì chỉ đầu ra có xác suất cao nhất.

Trong đề án, teacher là BGE-M3 gồm 568M tham số và student là MiniLM-L12 gồm 33M tham số. Student học cách BGE-M3 phân biệt mức relevance trên toàn bộ candidate list thay vì chỉ học nhãn relevant hoặc irrelevant.

Xu et al. (2025) cho thấy KD hiệu quả hơn contrastive learning thuần túy khi student đã có domain foundation phù hợp. Kết quả của đề án nhất quán với nhận định này: KD trực tiếp từ MiniLM base cho $\text{MRR}@10 = 0.5193$, thấp hơn base chưa huấn luyện ở mức 0.5869. Vì vậy, Stage A domain adaptation là bước cần thiết trước KD.

2.3.3 Các loss functions cho reranker KD

Đề án tập trung phân tích bốn nhóm loss cốt lõi, mỗi nhóm sử dụng một dạng teacher signal khác nhau. Các biến thể mở rộng và kết quả thực nghiệm được trình bày trong Chương 5.

Listwise KL Divergence.

Điểm teacher và student được chuẩn hóa thành phân phối xác suất trên toàn bộ n candidates bằng softmax với temperature τ :

$$P_i^T = \frac{\exp(s_i^T/\tau)}{\sum_j \exp(s_j^T/\tau)}, \quad P_i^S = \frac{\exp(s_i^S/\tau)}{\sum_j \exp(s_j^S/\tau)}$$

$$\mathcal{L}_{\text{KL}} = \sum_i P_i^T \log \frac{P_i^T}{P_i^S}$$

KL divergence đo khoảng cách giữa hai phân phối. Đây là listwise loss vì tối ưu toàn bộ danh sách thay vì từng cặp riêng lẻ.

Margin-MSE.

Loss học từ chênh lệch điểm tương đối giữa các cặp candidates:

$$\mathcal{L}_{\text{MMSE}} = \frac{1}{|P|} \sum_{(i,j) \in P} [(s_i^S - s_j^S) - (s_i^T - s_j^T)]^2$$

Student không cần khớp điểm tuyệt đối mà khớp hiệu điểm. Tuy nhiên, cross-encoder logits của teacher có range lớn. Trên tập train của đồ án, margin có mean = 4.13, std = 2.57 và khoảng giá trị lên tới 23.3 units, khiến gradient dễ bị chi phối bởi easy pairs có margin lớn.

RankNet với teacher soft labels.

RankNet (Burges et al., 2005) được mở rộng bằng cách dùng xác suất liên tục từ teacher:

$$P_{ij}^T = \sigma(s_i^T - s_j^T)$$

$$\mathcal{L}_{\text{RN}} = -\frac{1}{n^2 - n} \sum_{i \neq j} [P_{ij}^T \log P_{ij}^S + (1 - P_{ij}^T) \log(1 - P_{ij}^S)]$$

RankNet tối ưu $O(n^2)$ cặp cho mỗi query. Với 20 candidates, số cặp có thứ tự là 380, tạo tín hiệu phong phú nhưng vẫn có thể chịu ảnh hưởng từ scale của teacher logits.

ADR-MSE (Asymmetric Distillation Ranking MSE).

ADR-MSE là loss được đề xuất trong đồ án, sử dụng rank positions thay vì continuous scores làm teacher signal. Với rank $r_i \in \{0, \dots, n-1\}$, trong đó $r=0$ là vị trí tốt nhất, target được ánh xạ về $[-1, +1]$:

$$\tilde{r}_i = 1 - \frac{2r_i}{n-1}$$

Student logits được chuẩn hóa theo từng candidate list:

$$\tilde{s}_i = \frac{s_i^S - \mu_s}{\sigma_s + \epsilon}$$

$$\mathcal{L}_{\text{ADR}} = \frac{1}{n} \sum_{i=1}^n (\tilde{s}_i - \tilde{r}_i)^2$$

Table 2.3: Ví dụ target và student score trong ADR-MSE

Candidate	Teacher rank r_i	Target $\tilde{r}_i$	Student logit s_i	Score chuẩn hóa $\tilde{s}_i$
Doc A (gold)	0	+1.00	2.5	+0.8
Doc B	1	+0.33	1.2	+0.2
Doc C	2	-0.33	0.1	-0.3
Doc D	3	-1.00	-1.5	-0.7

Với ví dụ trong Bảng 2.3, loss bằng:

$$\frac{(0.8 - 1.0)^2 + (0.2 - 0.33)^2 + (-0.3 + 0.33)^2 + (-0.7 + 1.0)^2}{4} \approx 0.037$$

Rank position là tín hiệu ordinal và ổn định, không phụ thuộc vào thang điểm tuyệt đối của teacher. Phép ánh xạ tuyến tính về $[-1, +1]$ giúp các vị trí có target bị chặn và hạn chế scale mismatch giữa teacher và student.

2.4 Mô hình nhúng văn bản

2.4.1 Kiến trúc Transformer và BERT

Transformer (Vaswani et al., 2017) là nền tảng của BERT, PhoBERT, MiniLM và BGE-M3. Transformer encoder xử lý chuỗi token $x_1, x_2, \dots, x_n$ qua L lớp, trong đó mỗi lớp gồm Multi-Head Self-Attention và Feed-Forward Network.

Self-attention cho phép mỗi token tham chiếu đến tất cả token khác trong chuỗi:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

trong đó Q , K và V là các ma trận query, key và value được chiếu từ hidden states, còn d_k là số chiều của key vector. Cơ chế này giúp model nắm bắt quan hệ dài hạn giữa các token mà không phụ thuộc trực tiếp vào khoảng cách vị trí.

BERT (Bidirectional Encoder Representations from Transformers; Devlin et al., 2019) được pretrain chủ yếu bằng Masked Language Modeling và Next Sentence

Prediction. Đặc tính bidirectional cho phép mỗi token nhận context từ cả bên trái và bên phải. Hidden state của token [CLS] thường được dùng làm biểu diễn tổng hợp của toàn chuỗi cho các tác vụ phân loại.

2.4.2 Vietnamese_Embedding_v2 và BGE-M3

AITeamVN/Vietnamese_Embedding_v2 là mô hình embedding tiếng Việt được xây dựng trên nền BGE-M3 (Chen et al., 2024). BGE-M3 là mô hình đa ngôn ngữ hỗ trợ ba chế độ retrieval: dense bi-encoder, learned sparse weights và multi-vector theo hướng ColBERT. Vietnamese_Embedding_v2 được fine-tune tiếp trên corpus tiếng Việt để cải thiện khả năng hiểu từ vựng và ngữ nghĩa đặc thù.

Trong hệ thống đề án, Vietnamese_Embedding_v2 được dùng ở chế độ dense bi-encoder và tạo vector 1024 chiều được chuẩn hóa L2. Baseline trước fine-tune đạt $\text{Acc}@1 = 73.89\%$ trên internal test gồm 157 queries. Đây là điểm xuất phát cho giai đoạn fine-tune embedding ở v2.

2.4.3 MultipleNegativesRankingLoss

MultipleNegativesRankingLoss – MNR (Henderson et al., 2017) là loss nền tảng cho contrastive learning của embedding model. Với batch gồm N cặp query–positive, mỗi query phải phân biệt positive của nó với $N - 1$ in-batch negatives, tức positive của các query khác:

$$\mathcal{L}_{\text{MNR}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_i})/\tau)}{\sum_{j=1}^N \exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_j})/\tau)}$$

trong đó $\mathbf{e}_{q_i}$ và $\mathbf{e}_{p_i}$ là embedding của query và positive tương ứng, còn τ là temperature.

Table 2.4: Ví dụ in-batch negatives với batch size bằng 3

Query	Positive	In-batch negatives
q_1 : “nhiệt độ tối đa?”	p_1 : “max temp: 85°C”	p_2, p_3
q_2 : “áp suất vận hành?”	p_2 : “operating pressure: 3 bar”	p_1, p_3
q_3 : “công suất tiêu thụ?”	p_3 : “power consumption: 500W”	p_1, p_2

Với q_1 , model phải tạo $\cos(q_1, p_1)$ cao hơn $\cos(q_1, p_2)$ và $\cos(q_1, p_3)$. In-batch negatives không yêu cầu nhãn negative tường minh. Với batch size = 128 trong đề án, mỗi query nhận 127 negatives. Vì vậy, batch size lớn giúp tạo tín hiệu học khó và đa dạng hơn, nhưng đồng thời đòi hỏi GPU có đủ VRAM.

2.4.4 Matryoshka Representation Learning

Matryoshka Representation Learning – MRL (Kusupati et al., 2022) huấn luyện embedding model sao cho mỗi prefix của vector cũng là một biểu diễn hữu ích. Tên gọi lấy cảm hứng từ búp bê Matryoshka: vector 1024 chiều chứa prefix 512 chiều, và prefix này tiếp tục chứa biểu diễn 256 chiều.

MRL tính MNR loss đồng thời trên nhiều kích thước trong một forward pass:

$$\mathcal{L}_{\text{MRL}} = \sum_{m \in \mathcal{M}} w_m \cdot \mathcal{L}_{\text{MNR}}(f_m(\mathbf{e}_q), f_m(\mathbf{e}_p), \{f_m(\mathbf{e}_{p_j})\})$$

trong đó $\mathcal{M} = \{256, 512, 1024\}$, $f_m(\mathbf{x}) = \mathbf{x}[:m]$ là phép lấy m chiều đầu và $w_m = 1$ cho mọi kích thước.

Loss được tính trên cả ba kích thước rồi cộng lại, buộc model tổ chức thông tin sao cho các chiều đầu mang thông tin quan trọng. Nếu chỉ tối ưu vector 1024 chiều, model có thể phân bổ thông tin tùy ý và prefix ngắn không nhất thiết còn hữu ích.

Một checkpoint MRL hỗ trợ triển khai linh hoạt theo tài nguyên. Trong đồ án, embedding 256 chiều đạt $\text{Acc}@1 = 73.89\%$, bằng baseline gốc, trong khi index ChromaDB nhỏ hơn 4 lần và tìm kiếm nhanh hơn khoảng 4 lần so với 1024 chiều.

2.4.5 Curriculum Learning

Curriculum Learning (Bengio et al., 2009) là chiến lược tổ chức dữ liệu huấn luyện theo độ khó tăng dần. Nếu model gặp hard negatives quá sớm khi tham số chưa ổn định, gradient có thể nhiễu và model dễ hội tụ vào nghiệm kém.

Trong bài toán embedding retrieval, độ khó của negative được xác định theo nguồn gốc và mức gần với gold chunk:

- **Easy negatives:** synthesized negatives do GPT tạo bằng cách thay đổi entity, số liệu hoặc logic trong positive chunk. Boundary với positive tương đối rõ ràng.
- **Medium negatives:** extra synthesized negatives từ corpus thực tế, có phân phối domain rộng hơn.
- **Hard negatives:** các chunk thực nằm ở rank 4–15 trong kết quả retrieval, gần gold trong không gian vector nhưng không phải gold.

Pipeline của đồ án gồm Stage 1 với 1,040 synthesized triplets, Stage 2 với 3,840 triplets gồm synthesized và extra synthesized, và Stage 3 với 6,985 triplets gồm toàn bộ dữ liệu cộng mined negatives. Checkpoint của stage trước được dùng làm điểm khởi đầu cho stage sau, đồng thời learning rate giảm dần.

2.5 Mô hình ngôn ngữ lớn trong pipeline RAG

2.5.1 GPT-4o-mini

GPT-4o-mini (OpenAI, 2024) là phiên bản nhỏ và hiệu quả hơn của GPT-4o, phù hợp với các tác vụ không yêu cầu toàn bộ năng lực của model lớn. Model hỗ trợ context window lên đến 128K tokens.

Trong v1, GPT-4o-mini đảm nhận hai vai trò. Thứ nhất là answer generation: nhận top-5 chunks cùng câu hỏi trắc nghiệm và sinh đáp án A/B/C/D. Thứ hai là intent routing: phân loại câu hỏi thành `tra_cuu` hoặc `tinh_toan` trong pipeline Router-Summary. Mỗi lần gọi API cho routing bổ sung khoảng 800ms latency và phát sinh chi phí, do đó trở thành một bottleneck của v1.

Trong giai đoạn xây dựng dữ liệu huấn luyện v2, GPT-4o-mini còn được dùng làm GPT judge để xác định gold chunks và lọc synthesized negatives. Quy trình này được trình bày trong Chương 4.

2.5.2 Chain-of-Thought Prompting

Chain-of-Thought – CoT prompting (Wei et al., 2022) khuyến khích LLM thực hiện các bước suy luận trung gian trước khi đưa ra câu trả lời cuối. Thay vì chỉ yêu cầu đáp án, CoT prompt hướng model trích xuất dữ kiện, lập luận hoặc tính toán từng bước.

Trong pipeline Router-Summary, câu hỏi được phân loại là `tinh_toan` nhận CoT prompt, còn câu hỏi `tra_cuu` sử dụng direct prompt ngắn hơn. Kết quả v1 cho thấy CoT cải thiện accuracy trên nhóm `tinh_toan` từ 63.16% của Baseline lên 85.53% của Router-Summary.

2.6 Query Routing và phân loại Intent

2.6.1 LLM-based Routing

Query routing phân tích câu hỏi đầu vào để chọn chiến lược xử lý phù hợp. Câu hỏi tra cứu có thể dùng direct retrieval và prompt ngắn, trong khi câu hỏi tính toán cần reasoning theo từng bước. Routing đúng giúp tăng accuracy mà không áp dụng pipeline tốn kém cho mọi query.

Trong v1, GPT-4o-mini phân loại intent đúng 985/991 câu, tương ứng accuracy 99.4%. Tuy nhiên, mỗi API call bổ sung khoảng 800ms latency và chi phí inference, trong khi bài toán chỉ gồm hai lớp. Đây là trường hợp sử dụng model có năng lực lớn hơn nhiều so với yêu cầu của tác vụ.

2.6.2 PhoBERT và phân loại văn bản tiếng Việt

PhoBERT (Nguyen et al., 2020) là BERT được pretrain trên khoảng 20GB văn bản tiếng Việt, gồm dữ liệu báo chí và Wikipedia. PhoBERT sử dụng BPE tokenizer được học từ corpus tiếng Việt nên phù hợp hơn với từ ghép, từ láy và các dạng viết tắt trong tiếng Việt so với nhiều model đa ngôn ngữ.

Kiến trúc PhoBERT-base gồm 12 Transformer layers, hidden size 768, 12 attention heads và khoảng 135M tham số. Để fine-tune cho phân loại nhị phân, một classification head được đặt trên hidden state của token [CLS]:

$$\hat{y} = \text{softmax}(\mathbf{W}\mathbf{h}_{[\text{CLS}]} + \mathbf{b})$$

trong đó $\mathbf{h}_{[\text{CLS}]} \in \mathbb{R}^{768}$ và $\mathbf{W} \in \mathbb{R}^{2 \times 768}$.

So với GPT-4o-mini router, PhoBERT fine-tuned chạy local, không cần API call và có latency mục tiêu khoảng 1–3ms. Thách thức chính là class imbalance: `tra_cuu` chiếm 915/991 câu (92.3%), còn `tinh_toan` chỉ chiếm 76/991 câu (7.7%). Chiến lược xử lý imbalance được trình bày trong Chương 4.

2.7 Research Gap

Qua khảo sát tài liệu và thực nghiệm hệ thống v1, đề án xác định ba khoảng trống nghiên cứu làm cơ sở cho các đóng góp kỹ thuật trong những chương tiếp theo.

Gap 1 – Lightweight reranker cho tiếng Việt.

PhoRanker (Nguyen Ba, 2023) là một cross-encoder reranker tiếng Việt đã được công bố, gồm 135M tham số và đạt $\text{MRR}@10 = 0.7415$ trong thực nghiệm của đề án. Chưa có nhiều nghiên cứu áp dụng Knowledge Distillation để xây dựng reranker nhỏ hơn cho miền tài liệu kỹ thuật tiếng Việt, đặc biệt là phân tích các KD loss và teacher signal. Đề án giải quyết khoảng trống này thông qua các đóng góp C2, C3 và C4.

Gap 2 – Phân tích latency–cost–accuracy trong Vietnamese RAG.

Các hệ thống RAG thường tập trung báo cáo accuracy mà ít phân tích đồng thời latency và chi phí inference. Bên cạnh đó, nhiều công trình tối ưu từng thành phần riêng lẻ thay vì xem xét toàn bộ pipeline. Đề án đồng thời tối ưu reranker, embedding và intent routing theo điều kiện triển khai CPU, tiếp cận bài toán dưới góc nhìn tối ưu đa mục tiêu qua đóng góp C5.

Gap 3 – Thiếu cơ sở thực nghiệm cho quyết định tối ưu.

Nhiều giải pháp được lựa chọn từ đầu mà không xuất phát từ bottleneck đo được trên một hệ thống hoàn chỉnh. Đồ án thực hiện theo hướng xây dựng và đo lường v_1 trước, sau đó dùng kết quả thực nghiệm làm căn cứ cho từng quyết định tối ưu ở v_2 . Cách tiếp cận này giúp tập trung nguồn lực vào bottleneck thực tế và tạo cơ sở cho đóng góp C1.

CHƯƠNG 3. HỆ THỐNG RAG BAN ĐẦU (V1)

3.1 Vai trò của hệ thống v1 trong đồ án

Hệ thống RAG ban đầu, gọi là v1 trong đồ án, được xây dựng như một prototype để kiểm chứng khả năng áp dụng RAG cho bài toán hỏi đáp tài liệu kỹ thuật tiếng Việt. Mục tiêu của v1 không phải tạo ra hệ thống cuối cùng, mà là một bước phát triển trung gian nhằm trả lời ba câu hỏi thực tế: pipeline RAG cơ bản có hoạt động ổn định trên dữ liệu kỹ thuật tiếng Việt hay không; thành phần nào tạo ra bottleneck khi triển khai; và cần tối ưu ở đâu để hình thành đóng góp kỹ thuật rõ ràng.

Trong giai đoạn này, hệ thống được chạy thử trên 991 câu hỏi trắc nghiệm và 200 tài liệu PDF từ tập phát triển. Chính 991 câu hỏi này sau đó được dùng để xây dựng dữ liệu huấn luyện cho v2, gồm nhãn intent, cặp query–positive chunk và negative samples cho reranker và embedding. Vì vậy, kết quả trên 991 câu chỉ được xem là tín hiệu phát triển nội bộ, không phải kết quả đánh giá cuối cùng.

Tập kiểm thử độc lập gồm 343 câu hỏi từ 49 tài liệu PDF được giữ riêng cho giai đoạn đánh giá v2 và không được sử dụng để huấn luyện.

Table 3.1: Vai trò của các tập dữ liệu trong quy trình phát triển

Nhóm dữ liệu	Quy mô	Vai trò
Tập phát triển v1	991 câu hỏi, 200 tài liệu PDF	Chạy thử pipeline, quan sát lỗi, đo bottleneck và tạo dữ liệu huấn luyện v2
Dữ liệu huấn luyện v2	Xây dựng từ 991 câu hỏi	Huấn luyện reranker, embedding và intent classifier
Tập kiểm thử độc lập	343 câu hỏi, 49 tài liệu PDF	Chỉ dùng để đánh giá v2, không dùng để huấn luyện hoặc lựa chọn cấu hình

3.2 Quy trình tiền xử lý tài liệu

3.2.1 OCR và chuẩn hóa văn bản

Tập dữ liệu gồm các PDF kỹ thuật thuộc nhiều lĩnh vực như phần mềm kiểm thử, thiết bị y tế, ô tô, điện, hệ thống HVAC và hướng dẫn vận hành thiết bị. Các tài liệu được chuyển đổi sang Markdown bằng công cụ OCR. Quá trình này giữ lại cấu trúc heading và biểu diễn bảng dưới dạng HTML, cho phép bước chunking phân biệt bảng với văn bản thường.

Mỗi tài liệu được lưu dưới dạng file `main.md` trong thư mục riêng theo định danh `Public_XXX`. Định danh được giữ trong metadata của từng chunk để hỗ trợ truy xuất trực tiếp khi câu hỏi nêu rõ tài liệu nguồn, chẳng hạn `Public_021`.

Ví dụ, một bảng thông số sau OCR vẫn giữ được cấu trúc hàng–cột như Bảng 3.2.

Table 3.2: Ví dụ bảng thông số được bảo toàn sau OCR

Thông số	Giá trị	Đơn vị
Nhiệt độ vận hành tối đa	85	°C
Áp suất vận hành	3.0	bar
Công suất tiêu thụ	500	W

Việc bảo toàn cấu trúc này giúp `TableAwareChunker` xử lý đúng quan hệ giữa header và từng ô dữ liệu.

3.2.2 TableAwareChunker

Chunking quyết định đơn vị thông tin được đưa vào retrieval. Với tài liệu kỹ thuật, chunk quá lớn, chẳng hạn trên 1,000 từ, làm loãng thông tin; ngược lại, chunk quá nhỏ, chẳng hạn dưới 100 từ, có thể làm mất ngữ cảnh hoặc tách header khỏi dữ liệu bảng. `TableAwareChunker` giải quyết vấn đề này bằng hai luồng xử lý riêng.

Text chunking.

Văn bản thuần được chia theo paragraph với ngân sách `chunk_size=512` từ và overlap khoảng 100 từ ở cấp paragraph. Overlap hạn chế mất thông tin tại ranh giới giữa hai chunk. Mỗi chunk lưu `hierarchy_path`, tức chuỗi heading $H1 > H2 > H3$ dẫn tới đoạn văn, để bảo toàn ngữ cảnh cấu trúc.

Table chunking.

Mỗi bảng được chia theo cửa sổ 10 hàng với overlap 2 hàng. Header được thêm lại vào đầu mỗi chunk để giữ ý nghĩa của từng cột. Nội dung HTML sau đó được chuyển thành text có cấu trúc, ví dụ:

```
Cột: Thông số | Giá trị | Đơn vị
Dòng 1: Nhiệt độ vận hành tối đa | 85 | độ C
Dòng 2: Áp suất vận hành | 3.0 | bar
Dòng 3: Công suất tiêu thụ | 500 | W
```

Biểu diễn text phù hợp với embedding model hơn HTML thô vì model chủ yếu được pretrain trên ngôn ngữ tự nhiên.

Metadata của chunk.

Mỗi chunk lưu bốn trường: `document_id`, `hierarchy_path`, `chunk_type` và `chunk_index`. Metadata hỗ trợ lọc theo tài liệu, xây dựng `DocIndex` và truy vết nguồn gốc khi phân tích lỗi.

Quy trình `TableAwareChunker` có thể tóm tắt như sau:

1. Tách file `main.md` thành các segment text và table dựa trên thẻ HTML.
2. Với segment text, tách thành paragraph và gộp tới giới hạn `chunk_size`, đồng thời giữ overlap cho chunk kế tiếp.
3. Với segment table, parse danh sách hàng, chia thành cửa sổ 10 hàng với overlap 2 hàng và thêm lại header.
4. Chuyển từng segment thành text, sau đó gán metadata và chỉ số thứ tự chunk.

3.2.3 Xây dựng chỉ mục

Sau chunking, các chunk được đưa vào ba cấu trúc chỉ mục phục vụ những pipeline khác nhau.

ChromaDB.

ChromaDB lưu dense embeddings của các chunk. Embedding được tạo bằng `AITeamVN/Vietnamese_Embedding_v2` dưới dạng vector 1024 chiều chuẩn hóa L2 và được cache vào `cache_chunk_embeddings.pkl`, tránh phải encode lại khi khởi động hệ thống.

BM25Okapi.

BM25Okapi lập chỉ mục toàn bộ chunk text cho sparse retrieval. Thành phần này không cần GPU, có latency dưới khoảng 10ms/query và phù hợp với mã số, ký hiệu hoặc tên thiết bị mà dense model có thể chưa biểu diễn tốt.

DocIndex.

DocIndex là chỉ mục bổ sung dành cho Router-Summary, trong đó mỗi tài liệu được biểu diễn bằng một doc-level vector. Với tài liệu có ít nhất ba text chunk, vector được tính bằng TextRank weighted mean và blend 85/15 với table embedding. Tài liệu có một hoặc hai text chunk dùng tỉ lệ blend 70/30. Tài liệu chỉ có bảng sử dụng hierarchy weighted mean với trọng số theo độ sâu heading.

DocIndex thu hẹp không gian tìm kiếm từ toàn bộ corpus xuống top-3 tài liệu trước khi thực hiện retrieval cấp chunk.

3.3 Kiến trúc ba pipeline RAG

Trên nền tiền xử lý và chỉ mục chung, đề án xây dựng ba pipeline để xác định trade-off giữa độ đơn giản, latency, chi phí API và khả năng xử lý các loại câu hỏi.

3.3.1 Pipeline Baseline

Baseline thực hiện hybrid retrieval trực tiếp trên toàn corpus rồi đưa context vào LLM để sinh đáp án.

Bước 1 – Hybrid Retrieval.

BM25 top-10 và ChromaDB dense top-10 chạy độc lập. Kết quả được hợp nhất bằng RRF với $k = 60$ để tạo candidate pool.

Bước 2 – BGE Rerank.

Cross-encoder BAAI/bge-reranker-v2-m3 chấm điểm từng cặp query–candidate. Sau khi toàn bộ candidates được score, hệ thống giữ top-5 chunk có relevance cao nhất.

Bước 3 – Phát hiện Public ID.

Nếu câu hỏi chứa định danh tài liệu như `Public_021`, candidate pool được giới hạn trực tiếp vào các chunk của tài liệu đó trước khi rerank. Heuristic này hiệu quả trong nhiều trường hợp nhưng có thể sai nếu câu hỏi nhắc tới một Public ID trong khi thông tin cần trả lời nằm ở tài liệu khác.

Bước 4 – Answer Generation.

Top-5 chunk được ghép cùng câu hỏi và các lựa chọn A/B/C/D. GPT-4o-mini sinh đáp án với `max_tokens=16` và `temperature=0`; prompt yêu cầu trả lời trực tiếp bằng lựa chọn cuối cùng.

Baseline chỉ gọi LLM một lần, có latency trung bình thấp nhất là 1.07s/câu và không phát sinh format error. Hạn chế là câu tra cứu và câu tính toán được xử lý giống nhau.

3.3.2 Pipeline Fusion

Fusion giữ nguyên Phase 1 gồm hybrid retrieval và rerank của Baseline. Kết quả Phase 1 được cache trong `cache_prepared.pkl` để hai pipeline dùng chung. Fusion bổ sung hai bước ở Phase 2.

Extract.

LLM thứ nhất nhận top-5 chunk và trích xuất tối đa 300 từ chứa thông tin cần thiết cho câu hỏi, đồng thời giữ nguyên số liệu và thuật ngữ. Lệnh gọi này sử dụng `max_tokens=400`.

Answer.

Phần text đã trích xuất thay thế raw context trong prompt trả lời. Ý tưởng là context ngắn và tập trung hơn có thể hỗ trợ reasoning, đặc biệt với câu hỏi cần kết hợp nhiều con số.

Fusion gọi LLM hai lần và đạt latency 5.76s/câu, cao hơn Baseline khoảng 5.4 lần. Bước extract cũng có nguy cơ loại bỏ thông tin liên quan, đặc biệt với câu

multi-answer.

3.3.3 Pipeline Router-Summary

Router-Summary gồm bốn bước nối tiếp và xử lý câu `tra_cuu` khác với câu `tinh_toan`.

Bước 1 – Router.

GPT-4o-mini phân loại intent thành `tra_cuu` hoặc `tinh_toan`; regex đồng thời trích xuất Public ID. Router đạt accuracy 99.4%, tương ứng 985/991 câu. Sáu câu bị nhầm intent vẫn được trả lời đúng ở đầu ra cuối.

Bước 2 – DocIndexer.

Nếu không có Public ID, DocIndexer chọn top-3 tài liệu bằng cách kết hợp BM25 trên doc-level text và cosine similarity trên doc-level vector qua RRF. Query embedding được tính một lần và tái sử dụng ở bước chunk retrieval. Nếu có Public ID, hệ thống chuyển sang direct mode và bỏ qua DocIndexer.

Bước 3 – MultiDocRetriever.

Hybrid retrieval được thực hiện trong phạm vi top-3 tài liệu đã chọn. BGE-M3 sau đó rerank candidate pool để lấy top-5 chunk cuối cùng. Việc giới hạn phạm vi retrieval giúp giảm nhiễu từ các tài liệu không liên quan.

Bước 4 – AnswerGenerator.

Câu `tinh_toan` sử dụng Chain-of-Thought prompt để khuyến khích suy luận từng bước trước khi chọn đáp án. Câu `tra_cuu` sử dụng direct prompt ngắn gọn tương tự Baseline.

3.3.4 So sánh kiến trúc ba pipeline

Table 3.3: So sánh kiến trúc ba pipeline v1

Đặc điểm	Baseline	Fusion	Router-Summary
Retrieval scope	Toàn corpus	Toàn corpus	Top-3 tài liệu qua DocIndexer
Rerank	BGE-M3	BGE-M3, dùng chung Phase 1	BGE-M3
Phân loại intent	Không	Không	GPT-4o-mini
Số lần gọi LLM/câu	1	2, extract + answer	2, router + answer
Prompt theo intent	Không	Không	CoT cho <code>tinh_toan</code>
Latency đo được	1.07s	5.76s	Chưa đo chính xác
Format error	0	9 câu	5 câu

Ba pipeline dùng chung nền tảng chunking, indexing và reranker. Khác biệt chính nằm ở phạm vi retrieval, việc phân loại intent, số lần gọi LLM và chiến lược prompt.

3.4 Đánh giá thực nghiệm v1

3.4.1 Thiết lập thực nghiệm

Table 3.4: Cấu hình thực nghiệm v1 trên tập phát triển

Thành phần	Cấu hình
Tập chạy thử	991 câu hỏi trắc nghiệm tiếng Việt
Phân phối intent	tra_cuu: 915 câu (92.3%); tinh_toan: 76 câu (7.7%)
Câu multi-answer	22 câu (2.2%) có ít nhất hai đáp án đúng
Embedding model	AITeamVN/Vietnamese_Embedding_v2
Rerank model	BAAI/bge-reranker-v2-m3
LLM	GPT-4o-mini, temperature=0
Top-K sau rerank	5 chunks
Ground truth	Nhãn thủ công trong ans.md

Metric chính là accuracy theo exact match đáp án A/B/C/D, được tính tổng thể và theo intent. Câu multi-answer chỉ được tính đúng khi hệ thống chọn đủ tất cả đáp án đúng và không chọn thêm đáp án sai. Do 991 câu hỏi sau đó được dùng để xây dựng dữ liệu huấn luyện v2, các số liệu trong mục này chỉ mô tả hành vi của v1 trên tập phát triển.

3.4.2 Kết quả tổng thể và theo intent

Table 3.5: Accuracy tổng thể và theo intent của ba pipeline v1

Chỉ số	Baseline	Fusion	Router-Summary
Overall accuracy	909/991 (91.73%)	892/991 (90.01%)	916/991 (92.43%)
tra_cuu, 915 câu	861/915 (94.10%)	829/915 (90.60%)	851/915 (93.01%)
tinh_toan, 76 câu	48/76 (63.16%)	63/76 (82.89%)	65/76 (85.53%)
Multi-answer, 22 câu	15/22 (68.18%)	10/22 (45.45%)	12/22 (54.55%)
Format error	0	9 câu	5 câu
Latency trung bình/câu	1.07s	5.76s	Chưa đo

Router-Summary đạt accuracy tổng thể cao nhất, 92.43%, so với 91.73% của Baseline và 90.01% của Fusion. Chênh lệch lớn nhất nằm ở nhóm `tinh_toan`: Router-Summary đạt 85.53%, cao hơn Baseline 22.37 điểm phần trăm. Kết quả này đến từ sự kết hợp giữa DocIndexer, giúp thu hẹp candidate pool, và CoT prompt, hỗ trợ LLM suy luận từng bước.

Baseline đạt kết quả tốt nhất trên nhóm `tra_cuu`, ở mức 94.10%. Với câu tra cứu trực tiếp, pipeline đơn giản không có bước trung gian nên ít nguy cơ làm mất thông tin.

Fusion cải thiện nhóm `tinh_toan` thêm 19.73 điểm phần trăm so với Baseline nhưng giảm 3.50 điểm ở `tra_cuu` và 22.73 điểm ở nhóm multi-answer. Bước

extract có thể loại bỏ chi tiết cần thiết. Với latency cao hơn 5.4 lần nhưng accuracy tổng thể thấp hơn Baseline 1.72 điểm phần trăm, Fusion không tạo ra trade-off phù hợp.

3.4.3 Phân tích chất lượng retrieval

Để phân biệt lỗi retrieval với lỗi reasoning, toàn bộ câu sai của Baseline và Router-Summary được đánh giá bằng GPT-4o-mini judge. Với mỗi trường hợp, judge xác định liệu context đã retrieve có đủ để một người đọc suy ra đáp án đúng hay không.

Table 3.6: Phân tích nguồn gốc lỗi của Baseline và Router-Summary

Pipeline	Tổng câu sai	Context đủ, lỗi reasoning	Context thiếu, lỗi retrieval
Baseline	82	54 (65.9%)	28 (34.1%)
Router-Summary	75	46 (61.3%)	29 (38.7%)

Trong khoảng 61–66% câu sai, context đã chứa đủ thông tin nhưng GPT-4o-mini vẫn không suy ra đáp án đúng. Điều này cho thấy reasoning của LLM là một giới hạn quan trọng của v1. Cải thiện reranker và embedding chủ yếu tác động lên khoảng 34–39% lỗi do context bị thiếu.

3.4.4 Phân tích lỗi định tính

Trong 75 câu sai của Router-Summary, 27 câu được phân tích thủ công để xác định nguyên nhân cụ thể.

Table 3.7: Phân loại nguyên nhân lỗi trên 27 câu của Router-Summary

Nhóm nguyên nhân	Số câu	Tỉ lệ	Khả năng cải thiện kỹ thuật
Retrieval thất bại, chunk đúng không được kéo	10	37.0%	Có, thông qua embedding hoặc reranker
Câu hỏi mơ hồ trong dataset	8	29.6%	Cần cải thiện dataset
Corpus thiếu nội dung liên quan	7	25.9%	Cần bổ sung tài liệu
Lỗi reasoning của LLM	4	14.8%	Một phần, bằng prompt hoặc model mạnh hơn

Các nhân trong Bảng 3.7 có thể chồng lấp, chẳng hạn một câu vừa mơ hồ vừa gây lỗi reasoning. Vì vậy, số lượng và tỉ lệ ở các hàng không được cộng trực tiếp.

Nhóm retrieval thất bại gồm hai trường hợp: gold chunk nằm ngoài top-20 candidates, hoặc gold có trong candidate pool nhưng bị reranker xếp dưới chunk sai. Đây là nhóm có thể can thiệp trực tiếp bằng cách cải thiện embedding và reranker.

Câu hỏi mơ hồ thường yêu cầu xác định “nội dung chính” của một tài liệu mà không có một chunk duy nhất làm đáp án. Nhóm corpus thiếu gồm các câu hỏi về

thông tin không tồn tại trong tài liệu. Hai nhóm này chiếm 15/27 trường hợp được phân tích, tương ứng 55.5%, và phản ánh giới hạn dữ liệu thay vì riêng pipeline.

3.5 Xác định bottleneck

V1 đạt accuracy phát triển 92.43% với Router-Summary, nhưng phân tích chi tiết cho thấy ba bottleneck về latency, chi phí và chất lượng retrieval. Mỗi vấn đề gắn với một thành phần và dẫn đến một hướng tối ưu riêng trong v2.

3.5.1 Bottleneck latency: Reranker BGE-M3

BAAI/bge-reranker-v2-m3 gồm 568M tham số và là thành phần nặng nhất trong retrieval pipeline. Khi chạy trên CPU với 20 candidates, latency reranking đạt khoảng 147ms/query, chiếm hơn 90% latency của bước retrieval và reranking.

Table 3.8: So sánh latency của các reranker trên CPU

Model	Params	Latency/query	MRR@10
BGE-M3	568M	khoảng 147ms	0.8891
PhoRanker	135M	khoảng 18ms	0.7415
MiniLM-L12 base	33M	khoảng 22ms	0.5869

Các chỉ số MRR trong Bảng 3.8 được đo trên tập test reranker độc lập và được phân tích đầy đủ ở Chương 5. Bảng cho thấy giảm kích thước model giúp giảm yêu cầu RAM và thời gian tải, nhưng một MiniLM chưa huấn luyện có chất lượng thấp. Vì vậy, bài toán đặt ra là distill tri thức từ BGE-M3 sang model 33M tham số mà vẫn duy trì chất lượng ranking.

3.5.2 Bottleneck chi phí: GPT Intent Router

Router-Summary gọi GPT-4o-mini hai lần cho mỗi câu hỏi. Lần gọi intent classification bổ sung khoảng 800ms latency và chi phí API, dù tác vụ chỉ phân biệt hai lớp `tra_cuu` và `tinh_toan`. Sáu câu bị phân loại sai vẫn có đáp án cuối đúng, cho thấy routing không đòi hỏi một model có năng lực sinh ngôn ngữ lớn.

Với đơn giá giả định \$0.15 cho một triệu input tokens và trung bình 200 input tokens/query, một triệu query cần khoảng 200 triệu input tokens, tương ứng khoảng \$30 chỉ tính phần input của bước routing. Chi phí thực tế còn phụ thuộc output tokens và bảng giá API tại thời điểm triển khai. Một PhoBERT fine-tuned có thể chạy local với chi phí suy luận biên gần bằng không và loại bỏ phụ thuộc mạng.

3.5.3 Bottleneck chất lượng: Embedding chưa tối ưu domain

`Vietnamese_Embedding_v2` là embedding tiếng Việt mạnh nhưng chưa được tối ưu riêng cho miền tài liệu kỹ thuật của đề án. Trên internal test của component embedding gồm 157 queries và corpus 2,204 chunks, baseline đạt $\text{Acc}@1 = 73.89\%$

và $MRR@10 = 0.8259$. Như vậy, 26.11% query không có gold chunk ở rank 1 và cần reranker cải thiện thứ hạng.

Nếu gold chunk không xuất hiện trong top-20 candidates, reranker không thể khôi phục. Phân tích định tính cũng cho thấy retrieval thất bại là nhóm lỗi kỹ thuật quan trọng. Các câu hỏi về thông số, bảng số liệu, ký hiệu và thuật ngữ chuyên ngành đòi hỏi embedding hiểu quan hệ domain-specific thay vì chỉ ngữ nghĩa tiếng Việt tổng quát.

Ba bottleneck trên xác định ba hướng tối ưu của v2: xây dựng lightweight reranker bằng Knowledge Distillation để giảm latency; fine-tune PhoBERT để thay GPT router; và fine-tune embedding bằng MRL kết hợp curriculum learning để cải thiện retrieval. Các phương pháp này được trình bày trong Chương 4.

CHƯƠNG 4. CÁC PHƯƠNG PHÁP TỐI ƯU ĐỀ XUẤT (V2)

4.1 Tổng quan định hướng v2

Từ ba bottleneck được xác định trong Chương 3, hệ thống v2 được thiết kế theo nguyên tắc mỗi thay đổi kỹ thuật phải giải quyết một vấn đề đã được đo lường, thay vì tối ưu dựa trên giả định. Ba hướng tối ưu tương ứng được trình bày trong Bảng 4.1.

Table 4.1: Ánh xạ bottleneck của v1 sang giải pháp v2

Bottleneck v1	Giải pháp v2	Mục tiêu thiết kế
BGE-M3: 568M tham số, khoảng 147ms/query	Distill sang MiniLM-L12 33M tham số	Latency ≤ 25 ms, MRR@10 ≥ 0.75
GPT router: khoảng 800ms và phát sinh chi phí API	Fine-tune PhoBERT intent classifier	Accuracy $\geq 97\%$, latency < 5 ms
Embedding baseline: Acc@1 = 73.89%	Fine-tune bằng MRL và curriculum learning	Acc@1 tăng ít nhất 3 điểm phần trăm, MRR@10 tăng ít nhất 0.03

Ba hướng có thể được phát triển và đánh giá tương đối độc lập. Các mục tiếp theo trình bày thiết kế, dữ liệu huấn luyện và quy trình của từng hướng; kết quả đánh giá được tổng hợp trong Chương 5.

4.2 Lightweight Reranker via Knowledge Distillation

4.2.1 Motivation và thiết kế tổng quan

Mục tiêu của hướng thứ nhất là thay BGE-M3, gồm 568M tham số và có latency khoảng 147ms/query, bằng MiniLM-L12 gồm 33M tham số và có latency khoảng 22ms/query. MiniLM nhỏ hơn khoảng 17 lần nhưng vẫn sử dụng kiến trúc cross-encoder phù hợp với reranking và đã có foundation về passage ranking từ MS MARCO.

Thực nghiệm ban đầu cho thấy KD trực tiếp từ MiniLM base không hiệu quả: MRR@10 chỉ đạt 0.5193, thấp hơn MiniLM base chưa huấn luyện ở mức 0.5869. Nguyên nhân là domain gap giữa English MS MARCO và tài liệu kỹ thuật tiếng Việt. Student chưa có biểu diễn phù hợp cho đầu vào nên không tiếp nhận tốt ranking signal từ teacher.

Quan sát này nhất quán với Xu et al. (2025): KD chỉ hiệu quả khi student đã có domain foundation phù hợp. Vì vậy, đồ án sử dụng pipeline hai giai đoạn:

- **Stage A – Contrastive Learning:** domain adaptation MiniLM-L12 bằng dữ liệu tiếng Việt kỹ thuật và BCE loss.
- **Stage B – Knowledge Distillation:** từ checkpoint Stage A, distill ranking

signal của BGE-M3 để học mức relevance tinh tế hơn.

4.2.2 Pipeline xây dựng dữ liệu huấn luyện

Dữ liệu reranker được xây dựng qua năm giai đoạn có input và output riêng. Quy trình này tạo positive chunks đã xác minh, synthesized hard negatives và mined negatives từ kết quả ranking.

Giai đoạn 1 – Retrieve & Rerank.

Toàn bộ 991 câu hỏi được đưa qua hybrid retrieval gồm BM25, ChromaDB dense retrieval và RRF, sau đó BGE-M3 rerank. Mỗi query giữ 20 candidates kèm bge_score và thứ hạng. File retrieve_rerank_991.jsonl chứa tổng cộng 19,820 cặp query–chunk và là đầu vào cho các giai đoạn sau.

Giai đoạn 2 – GPT Gold Chunk Judge.

GPT-4o-mini nhận câu hỏi và top-5 candidates, sau đó phân loại từng chunk thành gold, partial hoặc irrelevant, kèm confidence high, medium hoặc low. Nếu không tìm thấy gold trong top-5, hệ thống mở rộng sang các vị trí 5–9.

Kết quả có 793 queries tìm được gold chunk với confidence đủ cao. Trong các trường hợp còn lại, 141 queries thuộc nhóm low_score_skip, tức top-1 BGE score nhỏ hơn 0.3, và 57 queries thuộc nhóm none. GPT judge giúp tránh việc mặc định top-1 của BGE-M3 là positive trong những trường hợp partial chunk được xếp trên gold chunk.

Giai đoạn 3 – Synthesize Hard Negatives.

Với mỗi query có gold chunk, GPT tạo các phiên bản gần giống positive về văn phong và cấu trúc nhưng thay đổi thông tin quyết định answerability:

- **entity_swap**: thay thực thể lỗi, ví dụ “Model X200” thành “Model X500”.
- **temporal_edit**: thay số liệu hoặc mốc thời gian.
- **logic_flip**: đảo điều kiện hoặc quan hệ nhân quả.
- **provenance_conflict**: thay phạm vi hoặc đối tượng áp dụng.

Ví dụ, positive “người lao động được hưởng 1/2 tháng lương cho mỗi năm làm việc nếu làm việc từ đủ 12 tháng” có thể được sửa thành negative “được hưởng 1/4 tháng lương nếu làm việc từ đủ 6 tháng”. Negative vẫn tự nhiên nhưng dẫn tới câu trả lời sai.

Mỗi negative qua ba tầng lọc: self-check trong prompt; một GPT judge độc lập; và BGE-M3 gap filter, loại mẫu có $\text{score}_{pos} - \text{score}_{neg} < 0.1$. Sau lọc, 405 trong 1,291 synthesized candidates được giữ lại.

Table 4.2: Thống kê synthesized negatives sau lọc

Edit type	Tổng sinh	Giữ lại	Tỉ lệ
entity_swap	599	195	32.6%
temporal_edit	606	183	30.2%
logic_flip	55	18	32.7%
provenance_conflict	31	9	29.0%
Tổng	1,291	405	31.4%

Tỉ lệ loại 68.6% phản ánh tiêu chuẩn nghiêm ngặt: chỉ các negative khó nhưng không trả lời được query mới được dùng cho reranker.

Giai đoạn 4 – Mine Negatives từ BGE Rankings.

Các chunk ở rank 4–20 được xem xét làm mined negatives. Gold và partial chunks bị loại. Chunk cùng document với positive chỉ được giữ nếu GPT xác nhận irrelevant để hạn chế false negatives. Score gap được dùng để phân loại: hard trong khoảng 0.1–0.4, medium trong khoảng 0.4–0.7, và easy lớn hơn 0.7.

Quy trình thu được 9,917 mined negatives, trong đó 913 hard và 1,676 medium được sử dụng cho tập reranker.

Table 4.3: Thống kê mined negatives theo nguồn

Nguồn	Số negatives	Tỉ lệ
cross_doc, khác tài liệu	9,475	95.5%
irr_gpt, cùng tài liệu và được GPT xác nhận irrelevant	442	4.5%
Tổng	9,917	100%

Giai đoạn 5 – Merge & Split.

Synthesized negatives sau lọc (405) và mined hard+medium negatives (2,589) được gộp thành 2,994 triplets. Dữ liệu được split theo query thay vì theo triplet để cùng một query không xuất hiện ở cả train và dev. Tỉ lệ 90/10 tạo 2,689 train triplets từ 439 queries và 305 dev triplets từ 50 queries.

Table 4.4: Tổng quan dataset huấn luyện reranker

Nguồn negative	Triplets	Hardness	Tỉ lệ
Synthesized hard negatives	405	hard	13.5%
Mined hard negatives	913	hard	30.5%
Mined medium negatives	1,676	medium	56.0%
Tổng	2,994	—	100%

4.2.3 Stage A: Contrastive Learning

Stage A fine-tune MiniLM-L12 bằng Binary Cross-Entropy trên domain triplets:

$$\mathcal{L}_A = \text{BCE}(\sigma(s_{\text{pos}}), 1) + \text{BCE}(\sigma(s_{\text{neg}}), 0)$$

$$\mathcal{L}_A = -\log \sigma(s_{\text{pos}}) - \log (1 - \sigma(s_{\text{neg}}))$$

trong đó s_{pos} và s_{neg} là logits của MiniLM cho positive và negative chunk.

Một quyết định quan trọng là không dùng mMARCO-VI trong cấu hình cuối. Thêm 50,000 mẫu mMARCO-VI làm MRR@10 giảm từ 0.7318 xuống 0.7194. Phân tích cho thấy 94.9% mẫu mMARCO là easy negatives theo BGE-M3, với mean margin = 8.91 so với 4.13 của domain data. Dữ liệu general quá dễ làm loãng tín hiệu từ domain hard negatives.

Table 4.5: Hyperparameters của Stage A

Tham số	Giá trị
Learning rate	2×10^{-5}
Batch size	32
Epochs	5
Early stopping patience	2
Seed	42
Optimizer	AdamW

Sau Stage A, MRR@10 đạt 0.7318, tăng 24.7% so với MiniLM base ở mức 0.5869. Checkpoint này tạo domain foundation cho Stage B.

4.2.4 Stage B: Knowledge Distillation

Stage B distill BGE-M3 vào checkpoint Stage A. Với mỗi query, teacher và student cùng score 20 candidates. Loss kết hợp KD signal và CL anchor:

$$\mathcal{L}_B = \alpha \mathcal{L}_{\text{KD}} + (1 - \alpha) \mathcal{L}_A, \quad \alpha = 0.7$$

Như vậy, KD có trọng số 0.7 và CL anchor có trọng số $1 - \alpha = 0.3$. Anchor giúp student duy trì domain discrimination đã học ở Stage A. KD thuần với $\alpha = 1.0$ cho MRR@10 = 0.7122, thấp hơn Stage A only ở mức 0.7318.

Listwise KL Divergence.

$$\mathcal{L}_{\text{KL}} = \sum_i P_i^T \log \frac{P_i^T}{P_i^S}, \quad P_i^T = \frac{\exp(s_i^T/\tau)}{\sum_j \exp(s_j^T/\tau)}, \quad P_i^S = \frac{\exp(s_i^S/\tau)}{\sum_j \exp(s_j^S/\tau)}$$

Với $\tau = 2.0$, loss tối ưu phân phối của toàn bộ 20 candidates. ListwiseKL nhạy với noisy queries; lọc còn 792 queries có gold rõ ràng làm MRR tăng từ 0.7226 lên 0.7430.

Margin-MSE.

$$\mathcal{L}_{\text{MMSE}} = \frac{1}{|P|} \sum_{(i,j) \in P} [(s_i^S - s_j^S) - (s_i^T - s_j^T)]^2$$

Margin-MSE học khoảng cách tương đối giữa các candidates. Tuy nhiên, teacher margins có mean = 4.13, std = 2.57 và range 23.3 units; 37% mẫu có margin ≥ 5.0 . Gradient từ easy pairs vì vậy có thể lớn hơn 25 lần so với hard negatives có margin 0.1–0.4.

RankNet với Teacher Soft Labels.

$$P_{ij}^T = \sigma(s_i^T - s_j^T), \quad P_{ij}^S = \sigma(s_i^S - s_j^S)$$

$$\mathcal{L}_{\text{RN}} = -\frac{1}{n^2 - n} \sum_{i \neq j} [P_{ij}^T \log P_{ij}^S + (1 - P_{ij}^T) \log(1 - P_{ij}^S)]$$

Với 20 candidates, RankNet tối ưu 380 cặp có thứ tự cho mỗi query. Tín hiệu pairwise phong phú nhưng vẫn chịu ảnh hưởng bởi scale của teacher logits.

ADR-MSE (Asymmetric Distillation Ranking MSE).

ADR-MSE là loss được đề xuất trong đề án. Thay vì dùng điểm teacher, loss sử dụng rank position $r_i \in \{0, \dots, n-1\}$ và ánh xạ tuyến tính về $[-1, +1]$:

$$\tilde{r}_i = 1 - \frac{2r_i}{n-1}$$

Student logits được chuẩn hóa theo candidate list:

$$\tilde{s}_i = \frac{s_i^S - \mu_s}{\sigma_s + \epsilon}$$

$$\mathcal{L}_{\text{ADR}} = \frac{1}{n} \sum_{i=1}^n (\tilde{s}_i - \tilde{r}_i)^2$$

Rank position là tín hiệu ordinal, không phụ thuộc vào thang điểm tuyệt đối của teacher. Phép ánh xạ tạo target bị chặn và cách đều; ADR-MSE gán trọng số bằng nhau cho các vị trí trong phiên bản cơ sở.

Table 4.6: Ví dụ tính ADR-MSE với năm candidates

Candidate	Rank r_i	Target $\tilde{r}_i$	Logit s_i	Score $\tilde{s}_i$	Loss item
Doc A (gold)	0	+1.00	3.2	+1.455	0.207
Doc B	1	+0.50	1.8	+0.712	0.045
Doc C	2	0.00	0.3	-0.085	0.007
Doc D	3	-0.50	-0.9	-0.722	0.049
Doc E	4	-1.00	-2.1	-1.359	0.129

Với mean logit $\mu_s = 0.460$ và population standard deviation $\sigma_s = 1.883$, ví dụ trên cho:

$$\mathcal{L}_{\text{ADR}} \approx \frac{0.207 + 0.045 + 0.007 + 0.049 + 0.129}{5} \approx 0.088$$

Table 4.7: Hyperparameters của Stage B

Tham số	Giá trị
KD weight α	0.7
Learning rate	1×10^{-5}
Batch size	16 queries, 20 candidates/query
ListwiseKL temperature τ	2.0
Epochs	3
Early stopping	Patience 2 theo dev MRR@10

4.3 Fine-tune Embedding bằng MRL và Curriculum Learning

4.3.1 Pipeline xây dựng dữ liệu embedding

Dữ liệu embedding sử dụng nền tảng từ pipeline reranker và bổ sung **extra synthesized data**. Nguồn này được GPT tạo trực tiếp từ ChromaDB corpus bằng cách thay từ hoặc số liệu quan trọng trong positive chunk. Nguồn extra synthesized không qua independent judge, do đó có chất lượng xác minh thấp hơn nhưng cung cấp phân phối đa dạng cho Stage 2.

Master dataset gồm 786 eligible queries và ba nguồn negative:

Table 4.8: Các nguồn negative trong master dataset embedding

Nguồn	Số negatives	Ghi chú
Synthesized candidates	1,291	Tập ứng viên được tạo trước bộ lọc nghiêm ngặt dành riêng cho reranker
Mined, rank 4–15 và tối đa 5/query	9,103	Chunk thật nằm gần gold trong không gian vector
Extra synthesized	3,476	GPT-generated, không có independent judge
Tổng	13,870	—

Việc dùng 1,291 synthesized records ở master embedding khác với 405 triplets nghiêm ngặt của reranker: 405 là tập vượt qua toàn bộ ba tầng lọc để huấn luyện cross-encoder, còn embedding curriculum sử dụng pool rộng hơn để tăng độ đa dạng.

Dữ liệu được split stratified theo intent với tỉ lệ 80/20, tạo 629 train queries và 157 internal test queries. Ba file curriculum gồm:

- `train_stage1.jsonl`: synthesized only, 1,040 triplets.
- `train_stage2.jsonl`: synthesized và extra synthesized, 3,840 triplets.
- `train_stage3.jsonl`: tất cả nguồn, mined tối đa 5/query, 6,985 triplets.

4.3.2 Matryoshka Loss và MNR

Base model là AITeamVN/Vietnamese_Embedding_v2, dựa trên BGE-M3 và tạo vector 1024 chiều. MultipleNegativesRankingLoss với batch size $N = 128$ cho mỗi query 127 in-batch negatives:

$$\mathcal{L}_{\text{MNR}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_i})/\tau)}{\sum_{j=1}^N \exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_j})/\tau)}$$

Trong thực nghiệm, scale = 20 tương ứng $\tau = 0.05$. MatryoshkaLoss tính MNR đồng thời tại ba kích thước $\mathcal{M} = \{256, 512, 1024\}$:

$$\mathcal{L}_{\text{MRL}} = \sum_{m \in \mathcal{M}} \mathcal{L}_{\text{MNR}}(f_m(\mathbf{e}_q), f_m(\mathbf{e}_p), \{f_m(\mathbf{e}_{p_j})\})$$

trong đó $f_m(\mathbf{x}) = \mathbf{x}[:m]$. Ba loss thành phần có trọng số bằng nhau. Batch size 128 cung cấp tín hiệu phong phú hơn batch size 16, nhưng cần GPU có VRAM lớn để duy trì model 1024 chiều.

4.3.3 Curriculum Training ba giai đoạn

Ba stage được train nối tiếp. Checkpoint stage trước là điểm khởi đầu của stage sau và learning rate giảm dần khi độ khó tăng.

Table 4.9: Cấu hình curriculum training cho embedding

Stage	Nguồn dữ liệu	Triplets	Epochs	LR	Warmup steps
Stage 1 – Easy	Synthesized only	1,040	2	2×10^{-6}	50
Stage 2 – Medium	Synth + Extra synth	3,840	2	1×10^{-6}	10
Stage 3 – Hard	Tất cả + mined	6,985	1	5×10^{-7}	5

Các tham số chung gồm AdamW, `weight_decay=0.01`, `max_grad_norm=1.0`, mixed precision và `max_seq_length=512`.

Stage 1 dùng synthesized negatives có boundary rõ ràng. Stage 2 thêm extra synthesized để mở rộng phân phối domain. Stage 3 thêm mined negatives là các chunk thật nằm gần gold trong không gian vector. Trình tự này hạn chế việc model phải học hard negatives ngay khi biểu diễn chưa ổn định.

Một hạn chế được phát hiện sau thực nghiệm là 50 warmup steps của Stage 1 quá dài so với khoảng 9 steps/epoch, do $1,040/128 \approx 9$. Model gần như không đạt full learning rate trong Stage 1. Đây là tham số cần giảm xuống khoảng 5–10 steps trong các thử nghiệm tiếp theo.

4.3.4 Chiến lược deployment với MRL

Một checkpoint hỗ trợ ba cấu hình:

Table 4.10: Các chế độ deployment từ một checkpoint MRL

Dimension	Cách sử dụng	Mục tiêu
256	<code>truncate_dim=256</code>	Tìm kiếm nhanh, index nhỏ hơn 4 lần
512	<code>truncate_dim=512</code>	Cân bằng chất lượng và tài nguyên
1024	Kích thước mặc định	Chất lượng retrieval cao nhất

Chiến lược two-stage dùng vector 256 chiều để search top-50 trên toàn index, sau đó tính lại cosine similarity ở 1024 chiều trong 50 candidates để chọn top-k cuối. Cách này giảm kích thước index và chi phí search toàn corpus, trong khi bước re-score vẫn tận dụng biểu diễn đầy đủ.

4.4 Fine-tune PhoBERT Intent Classifier

4.4.1 Kiến trúc và dữ liệu

PhoBERT-base-v2 được fine-tune cho hai lớp `tra_cuu` và `tinh_toan`. Classification head `Linear(768, 2)` được đặt trên hidden state của token `[CLS]`:

$$\mathbf{h}_{[\text{CLS}]} = \text{PhoBERT}(\text{tokenize}(q))_{[\text{CLS}]} \in \mathbb{R}^{768}$$

$$\hat{y} = \text{softmax}(\mathbf{W}\mathbf{h}_{[\text{CLS}]} + \mathbf{b}) \in \mathbb{R}^2$$

trong đó $\mathbf{W} \in \mathbb{R}^{2 \times 768}$ và $\mathbf{b} \in \mathbb{R}^2$.

Dataset gồm 991 câu hỏi có nhãn intent được phân loại thủ công và xác minh. Split stratified theo group với tỉ lệ 70/15/15 tạo 694 câu train, 148 câu validation và 149 câu internal test.

Ví dụ, “Theo tài liệu `Public_021`, nhiệt độ vận hành tối đa là bao nhiêu?” thuộc lớp `tra_cuu`; câu hỏi tính chi phí nhiên liệu từ quãng đường, mức tiêu hao và giá xăng thuộc lớp `tinh_toan`.

4.4.2 Xử lý class imbalance

`tra_cuu` chiếm 915/991 câu (92.3%), còn `tinh_toan` chỉ có 76/991 câu (7.7%). Một classifier luôn dự đoán lớp đa số đã có accuracy 92.3%, do đó accuracy riêng lẻ là metric dễ gây hiểu nhầm. F1-macro được dùng để đánh giá bình đẳng hai lớp.

Class weights.

CrossEntropyLoss sử dụng trọng số tỉ lệ nghịch với tần suất lớp và được chuẩn hóa:

$$w_{\text{tra_cuu}} = \frac{1/0.923}{1/0.923 + 1/0.077} \approx 0.077$$

$$w_{\text{tinh_toan}} = \frac{1/0.077}{1/0.923 + 1/0.077} \approx 0.923$$

Lỗi trên lớp `tinh_toan` vì vậy bị phạt mạnh hơn.

WeightedRandomSampler.

Xác suất lấy mẫu tỉ lệ nghịch với tần suất lớp, giúp mỗi batch gần cân bằng thay vì giữ tỉ lệ tự nhiên 92/8. Với batch size 32, nếu không có sampler thì trung bình

chỉ có khoảng hai mẫu `think_toan` trong mỗi batch.

Table 4.11: Hyperparameters fine-tune PhoBERT

Tham số	Giá trị
Learning rate	2×10^{-5}
Batch size	32
Epochs	15
Warmup ratio	0.1
Weight decay	0.01
Early stopping	Theo validation F1-macro
Optimizer	AdamW

Early stopping theo F1-macro tránh dừng sớm khi accuracy tổng thể cao nhưng recall của lớp thiểu số còn thấp.

4.4.3 Routing logic trong pipeline v2

Trong v1, query được gửi tới GPT-4o-mini qua API để nhận intent label, bổ sung khoảng 800ms. Trong v2, query được tokenize và chạy qua PhoBERT local, với latency mục tiêu khoảng 1–3ms. Logic sau classification không đổi: `tra_cuu` dùng direct prompt, còn `think_toan` dùng Chain-of-Thought prompt. Regex trích xuất Public ID chạy song song với classifier.

Table 4.12: So sánh GPT router của v1 và PhoBERT classifier của v2

Tiêu chí	GPT-4o-mini v1	PhoBERT v2
Accuracy	99.4% (985/991)	TBD
F1-macro	—	TBD
Latency/query	khoảng 800ms	khoảng 1–3ms
Chi phí/1M queries	khoảng \$30 tiền input theo giá định 200 tokens/query	\$0 chi phí API
Offline	Không	Có
Model	API remote	135M tham số, local

PhoBERT local loại bỏ phụ thuộc vào external API và giữ dữ liệu câu hỏi trong hạ tầng nội bộ, phù hợp với yêu cầu bảo mật của môi trường doanh nghiệp.

CHƯƠNG 5. THỰC NGHIỆM VÀ ĐÁNH GIÁ

5.1 Thiết lập thực nghiệm

5.1.1 Môi trường và phần cứng

Toàn bộ quá trình huấn luyện và đánh giá được thực hiện trên hai môi trường. Huấn luyện reranker và embedding sử dụng GPU NVIDIA RTX 6000 Ada (48GB VRAM), cấu hình đủ để duy trì batch size lớn cho MNR (128) và chạy teacher BGE-M3 song song với student MiniLM trong Stage B. Một số thử nghiệm bổ sung sử dụng Kaggle P100 (16GB).

Đánh giá inference và đo latency được thực hiện trên CPU để phản ánh đúng điều kiện triển khai thực tế. Đây là lý do chính để đề án lựa chọn mục tiêu tối ưu latency CPU thay vì chỉ tối ưu trên GPU.

5.1.2 Tập dữ liệu đánh giá

Đề án sử dụng hai tập test cho hai bài toán khác nhau và một tập dev để theo dõi quá trình huấn luyện. Tập 343 câu hỏi trên 49 PDF unseen là tập đánh giá zero-shot chính thức cho khả năng tổng quát hóa của reranker và pipeline v2. Tập 157 queries là internal test riêng cho component embedding, được dùng để so sánh các cấu hình embedding trong quá trình phát triển.

Table 5.1: Các tập dữ liệu dùng trong đánh giá

Tập	Quy mô	Mục đích
Test reranker	343 câu hỏi, 49 PDF unseen	Đánh giá zero-shot reranker và khả năng generalization trên tài liệu mới
Internal test embedding	157 queries, 2,204 chunks	So sánh các cấu hình embedding retrieval trong quá trình phát triển
Dev reranker	305 triplets, 50 queries	Theo dõi và lựa chọn checkpoint trong quá trình huấn luyện

Tập test reranker (343 câu, 49 PDF) hoàn toàn tách biệt với tập phát triển v1 (991 câu, 200 PDF), không có tài liệu nào trùng lặp. Đây là điều kiện zero-shot: model chưa từng thấy bất kỳ chunk nào từ 49 PDF này trong quá trình huấn luyện. Kết quả trên tập này do đó phản ánh khả năng generalization của model, không phải memorization.

5.1.3 Metrics đánh giá

Đánh giá reranker sử dụng bốn metrics tiêu chuẩn trong information retrieval: Hit@1 (tỉ lệ gold chunk xuất hiện ở rank 1), Recall@5 (tỉ lệ gold chunk xuất hiện trong top-5), MRR@10 (Mean Reciprocal Rank, đo vị trí trung bình của gold

chunk) và NDCG@10 (Normalized Discounted Cumulative Gain, đo chất lượng toàn bộ thứ tự xếp hạng). Metric chính để so sánh là MRR@10 và Recall@5 vì chúng phản ánh trực tiếp hiệu quả của reranker trong pipeline RAG thực tế, nơi top-5 chunks được đưa vào LLM.

Đánh giá embedding.

Các metrics gồm Acc@1 (tương đương Hit@1), Acc@10 và MRR@10 , được đo trên internal test gồm 157 queries.

Đánh giá end-to-end sử dụng accuracy trên 343 câu hỏi trắc nghiệm (exact match với đáp án A/B/C/D), kết hợp đo latency (ms/query) và chi phí API.

5.2 Đánh giá Reranker

5.2.1 Ablation study: xác nhận vai trò của Stage A

Nhóm thực nghiệm đầu tiên kiểm tra vai trò của Stage A (Contrastive Learning) trước khi áp dụng Knowledge Distillation, đồng thời đánh giá ảnh hưởng của hệ số α trong Stage B.

Table 5.2: Ablation về vai trò của Stage A và hệ số α

#	Cấu hình	Params	Recall @5	MRR @10	NDCG @10	Hit@1
1	MiniLM-L12 base (no training)	33M	0.7347	0.5869	0.6536	0.4694
2	KD trực tiếp từ base (không có Stage A)	33M	0.7347	0.5193	—	—
3	Stage A only (không dùng mMARCO)	33M	0.8746	0.7318	0.7801	0.6181
4	Stage A + KD only, $\alpha = 1.0$ (ListwiseKL)	33M	0.8484	0.7122	—	—
5	Stage A + KD only, $\alpha = 1.0$ (ADR-MSE)	33M	0.9038	0.7529	0.7960	0.6501
6	Stage A + ADR-MSE, $\alpha = 0.7$ (KD+CL)	33M	0.9096	0.7827	0.8222	0.6822

Cấu hình #2, KD trực tiếp từ base, cho $\text{MRR@10} = 0.5193$, thấp hơn cả MiniLM base chưa huấn luyện (#1: 0.5869). Khoảng cách miền giữa English MS MARCO, nguồn pretrain của MiniLM, và tài liệu kỹ thuật tiếng Việt quá lớn; student chưa có nền biểu diễn phù hợp nên KD không hội tụ hiệu quả. Stage A ở cấu hình #3 giải quyết vấn đề này và nâng MRR lên 0.7318, tương ứng mức cải thiện 24.7% so với base trước khi áp dụng KD. Kết quả này nhất quán với nhận định của Xu et al. (2025): KD chỉ phát huy hiệu quả khi student đã có foundation phù hợp với miền dữ liệu.

So sánh #4 và #5 cho thấy lựa chọn loss ảnh hưởng lớn ngay cả khi chỉ dùng KD ($\alpha = 1.0$): ADR-MSE đạt $\text{MRR} = 0.7529$, vượt ListwiseKL ở mức 0.7122 nhờ tín hiệu thứ hạng ổn định hơn. So sánh #5 và #6 tiếp tục xác nhận CL anchor vẫn cần thiết với ADR-MSE. Việc kết hợp KD và CL tại $\alpha = 0.7$ cải thiện thêm 0.0298 MRR so với KD thuần.

5.2.2 Ablation study: ảnh hưởng của mMARCO-VI

Nhóm thực nghiệm thứ hai kiểm tra ba cách tích hợp 50,000 mẫu tiếng Việt tổng quát từ mMARCO-VI vào pipeline huấn luyện.

Table 5.3: So sánh các hướng tích hợp mMARCO-VI

#	Cấu hình	Recall @5	MRR @10	NDCG @10	Ghi chú
7	Stage A+B ListwiseKL (gộp mMARCO)	0.8601	0.7043	0.7548	mMARCO làm loãng tín hiệu domain
8	Stage A+B ListwiseKL (không mMARCO)	0.8688	0.7430	0.7884	Chỉ dùng domain data, tốt hơn #7
9	Ba stage A1(mMARCO) → A2(domain) → B, chỉ đánh giá A2	0.8863	0.7293	0.7781	A2 gần tương đương Stage A không mMARCO
10	Ba stage A1(mMARCO) → A2(domain) → B, đầy đủ	0.9125	0.7729	0.8153	Kém hơn ADR-MSE hai stage
6	Stage A+B ADR-MSE, không mMARCO, hai stage	0.9096	0.7827	0.8222	Tốt nhất trong nhóm này

Cả ba hướng tích hợp đều cho kết quả kém hơn pipeline hai stage không dùng mMARCO (#6). Khi gộp chung dữ liệu ở cấu hình #7, 94.9% mẫu mMARCO được BGE-M3 đánh giá là easy, với mean logit margin = 8.91 so với 4.13 của tập domain train. Số lượng lớn easy samples vì vậy làm loãng tín hiệu từ các hard negatives thuộc miền kỹ thuật.

Với pipeline tuần tự ba stage (#9 và #10), Stage A2 trên dữ liệu domain ghi đề gần như hoàn toàn kiến thức học ở Stage A1 do catastrophic forgetting. Kết quả A2 only đạt MRR = 0.7293, gần bằng Stage A không dùng mMARCO (0.7318), cho thấy đóng góp của A1 hầu như bị mất. Hard-mining từ mMARCO cũng được thử nghiệm nhưng mean margin vẫn đạt 7.09, cao hơn đáng kể so với domain train (4.13), nên không cải thiện chất lượng tín hiệu.

Như vậy, với 2,689 mẫu hard và medium thuộc đúng miền dữ liệu, mMARCO-VI không mang lại lợi ích dù được tích hợp theo nhiều cách. Kết quả cho thấy tín hiệu từ một tập domain-specific nhỏ nhưng chất lượng cao có giá trị hơn một tập general data lớn và nhiều easy negatives.

5.2.3 So sánh các loss functions và teacher models trong Stage B

Nhóm thực nghiệm thứ ba so sánh các loss functions và teacher models trên cùng checkpoint Stage A không dùng mMARCO, với cùng siêu tham số $\alpha = 0.7$ và seed = 42.

Table 5.4: So sánh các loss functions và teacher models trong Stage B

Loss / Teacher	Teacher signal	Data	Recall@5	MRR@10	NDCG@10	Hit@1
ListwiseKL	Score distribution	792 queries	0.8688	0.7430	0.7884	0.6414
Margin-MSE	Score margins	2,202 pairs	0.8805	0.7372	0.7832	0.6356
RankNet	Pairwise probabilities	991 queries	0.8717	0.7419	0.7858	0.6443
ADR-MSE, BGE-M3 teacher	Rank positions	991 queries	0.9096	0.7827	0.8222	0.6822
Discounted ADR-MSE	Rank + NDCG weights	991 queries	0.8834	0.7426	0.7873	0.6297
Top-K ADR-MSE ($k = 10$)	Top-10 ranks only	991 queries	0.8717	0.7350	0.7781	0.6152
ADR-MSE, Jina-v3 teacher	Rank positions Jina	983 queries	0.8980	0.7694	0.8120	0.6706
ADR-MSE, Voyage 2.5 top-50	Rank positions Voyage	983 queries	0.8980	0.7715	0.8126	0.6764
ADR-MSE, Voyage 2.5 top-20	Rank positions Voyage	983 queries	0.9155	0.7925	0.8329	0.6997

ADR-MSE với BGE-M3 teacher vượt ListwiseKL 0.0397 MRR, Margin-MSE 0.0455 và RankNet 0.0408. Hai biến thể Discounted ADR-MSE và Top-K ADR-MSE đều kém hơn ADR-MSE gốc, cho thấy việc gán trọng số bằng nhau cho toàn bộ 20 vị trí phù hợp hơn với tập huấn luyện nhỏ gồm 983–991 queries.

Margin-MSE học từ hiệu điểm cross-encoder logit của teacher, $\Delta T_{ij} = T_i - T_j$, có thang đo không bị chặn. Trên 2,689 mẫu train, margin có mean = 4.13, std = 2.57, khoảng giá trị 23.3 units và 37% mẫu có margin ≥ 5.0 . Gradient từ các mẫu margin lớn có thể cao hơn 25 lần so với hard negatives có margin 0.1–0.4, khiến loss bị chi phối bởi easy pairs. ADR-MSE chuyển rank position về target chuẩn hóa trong $[-1, +1]$, nhờ đó tránh scale mismatch và giữ độ lớn gradient cân bằng hơn giữa các vị trí.

Thử nghiệm với Jina-v3 (MRR@10 = 0.9195 khi đánh giá trực tiếp) cho student kém hơn BGE-M3 teacher: MRR = 0.7694 so với 0.7827. Jina-v3 là listwise model phải chia nhỏ 20 candidates thành các cửa sổ 5 tài liệu, không có so sánh trực tiếp giữa các cửa sổ và tạo ranking noise khi chuyển sang rank positions. Voyage Rerank 2.5 (MRR@10 = 0.9381 khi đánh giá trực tiếp) được thử nghiệm với hai cấu hình candidate pool. Với top-50 candidates (MRR student = 0.7715), kết quả kém hơn BGE-M3 do chuẩn hóa ADR-MSE bị lỗi: với $n = 50$, các rank 0–10 đều nhận tín hiệu dương nằm trong khoảng $[+1.00, +0.59]$, gradient phân biệt relevant và non-relevant yếu hơn so với $n = 20$ ($[+1.00, -0.05]$). Ngoài ra, 30 candidates thêm ở vị trí 21–50 chủ yếu là easy negatives rõ ràng, không đóng góp tín hiệu học hữu ích. Khi thu hẹp về top-20 candidates, Voyage teacher đạt MRR student = 0.7925, vượt BGE-M3 teacher (+0.0098 MRR). Kết quả này cho thấy teacher mạnh hơn thực sự cải thiện student khi candidate pool được kiểm soát phù hợp: kích thước pool ảnh hưởng đến chất lượng tín hiệu ADR-MSE, không phải bản thân kiến trúc teacher.

5.2.4 So sánh với baselines

Table 5.5: So sánh MiniLM-ADR-MSE với các baselines

Model	Params	Hit@1	Recall@5	MRR@10	NDCG@10	Latency /query
MiniLM-L12 base	33M	0.4694	0.7347	0.5869	0.6536	khoảng 22ms
PhoRanker (VI)	135M	0.6472	0.8659	0.7415	0.7846	khoảng 18ms
MiniLM + ADR-MSE, BGE teacher	33M	0.6822	0.9096	0.7827	0.8222	khoảng 22ms
MiniLM + ADR-MSE, Voyage teacher	33M	0.6997	0.9155	0.7925	0.8329	khoảng 22ms
BGE-M3 (teacher)	568M	0.8309	0.9650	0.8891	0.9111	khoảng 147ms
Voyage Rerank 2.5 (API)	–	0.9009	0.9854	0.9381	0.9246	API

MiniLM + ADR-MSE với Voyage teacher (33M) là cấu hình tốt nhất trong số các student models, vượt PhoRanker (135M) trên cả bốn metrics với model nhỏ hơn 4 lần: +0.0525 Hit@1, +0.0496 Recall@5, +0.0510 MRR@10 và +0.0483 NDCG@10. So với BGE-M3 teacher, model đạt 84.2% Hit@1, 94.9% Recall@5 và 89.1% MRR@10 với số tham số nhỏ hơn 17 lần và latency thấp hơn 6.7 lần (22ms so với 147ms). Voyage Rerank 2.5 khi dùng trực tiếp qua API đạt MRR@10 = 0.9381, là upper bound thực nghiệm trên tập test; tuy nhiên, đây là dịch vụ cloud với latency mạng và chi phí per-request, không phù hợp với yêu cầu triển khai local của đề án.

Table 5.6: Đánh giá end-to-end các reranker trên 343 câu hỏi trắc nghiệm

Model	Đúng/Tổng	Accuracy	Latency/query
BGE-M3 (teacher)	333/343	97.1%	khoảng 17,249ms
MiniLM + ADR-MSE (ours)	326/343	95.0%	khoảng 1,621ms
MiniLM + ListwiseKL	325/343	94.8%	khoảng 1,438ms
MiniLM base	295/343	86.0%	khoảng 1,016ms

Khi thay riêng thành phần reranker và giữ nguyên các bước còn lại của quy trình đánh giá MCQ, ADR-MSE đạt accuracy 95.0%, chỉ thấp hơn BGE-M3 2.1 điểm phần trăm, tương ứng 7 câu, nhưng nhanh hơn khoảng 10.6 lần (1,621ms so với 17,249ms/query). Bảng 5.6 đánh giá ảnh hưởng của từng reranker trong cùng một cấu hình; đây chưa phải phép so sánh đầy đủ giữa toàn bộ pipeline v1 và v2 ở Mục 5.5.

5.2.5 Phân tích lỗi reranker

Best model (ADR-MSE với Voyage teacher, top-20) đạt $\text{Recall}@5 = 0.9155$, tức 29/343 queries không có gold chunk trong top-5. Phân tích dưới đây dựa trên cấu hình ADR-MSE với BGE-M3 teacher ($\text{Recall}@5 = 0.9096$, 31 miss cases) do có đầy đủ thông tin chi tiết từ quá trình phân tích; hai cấu hình có profile lỗi tương tự nhau.

Table 5.7: Phân tích 31 miss cases của ADR-MSE (BGE-M3 teacher)

Loại lỗi	Số lượng	Tỉ lệ	Mô tả
Near miss (gold rank 6–10)	15	48.4%	Gold chunk nằm ngay ngoài top-5
Retrieval miss (gold rank N/A)	4	12.9%	Gold không có trong top-20, reranker không thể khôi phục
Cross-domain noise	khoảng 13	41.9%	BM25/embedding match sai document hoặc sai domain
Same-domain confusion	khoảng 14	45.2%	Đúng domain nhưng chọn sai chunk cụ thể

Nhóm near miss (48.4%) phản ánh giới hạn của ngưỡng top-5: nếu pipeline đưa top-6 hoặc top-7 chunk vào LLM, một phần trong 15 queries này có thể được khôi phục. Tuy nhiên, việc tăng top-K cũng làm context dài hơn và tăng chi phí generation.

Nhóm retrieval miss gồm 4 trường hợp hoàn toàn ngoài tầm kiểm soát của reranker vì gold chunk không nằm trong top-20 candidates: (i) lệnh Offset trong CAD, tên lệnh ngắn khiến BM25 match sai tài liệu; (ii) hệ thống làm mát, gold nằm trong bảng checklist bảo dưỡng nhưng embedding không biểu diễn tốt; (iii) acronym UPS, truy xuất nhầm tài liệu hàng không có cùng từ viết tắt; (iv) ống soi dạ dày, hai chunk liền kề cùng tài liệu có surface form tương tự và retriever lấy nhầm chunk.

Các domain bị miss nhiều nhất gồm y tế/thiết bị y tế (5 cases), điện/vật liệu (4 cases), CAD/Autodesk (3 cases) và HVAC/Chiller (3 cases). Đây đều là các domain có nhiều bảng số liệu kỹ thuật và thuật ngữ đặc thù, nơi cả embedding lẫn BM25 gặp khó khăn khi exact-match thấp.

5.3 Đánh giá Embedding

5.3.1 Kết quả theo từng curriculum stage

Table 5.8: Tiến trình cải thiện qua 3 curriculum stages

Stage	Nguồn data	Triples	Acc@1	MRR@10	Delta Acc@1
Baseline (trước train)	–	–	73.89%	0.8259	–
Stage 1 – Easy	Synthesized negatives	1,040	74.52%	0.8313	+0.63%
Stage 2 – Medium	Synth + Extra synth	3,840	76.43%	0.8521	+2.54%
Stage 3 – Hard	Tất cả + mined	6,985	78.34%	0.8673	+4.45%

Mỗi stage đều cải thiện so với stage trước, xác nhận curriculum learning đúng hướng: model học boundary dễ trước khi tiếp cận hard negatives. Stage 2 cải thiện mạnh nhất (+2.54% Acc@1) nhờ extra synthesized mở rộng phân phối dữ liệu training từ 1,040 lên 3,840 triples. Stage 3 thêm mined negatives, tức các chunk thực tế nằm gần gold trong không gian vector, tiếp tục cải thiện Acc@1 thêm 1.91 điểm phần trăm.

Nếu train trực tiếp trên toàn bộ dữ liệu mà không áp dụng curriculum, model phải học hard negatives ngay từ đầu khi tham số chưa ổn định và có nguy cơ rơi vào local minima. Curriculum giảm rủi ro này bằng cách khởi động với synthesized negatives có boundary rõ ràng trước.

5.3.2 Ablation theo dimension với MRL

Table 5.9: Kết quả MRL theo kích thước embedding

Dimension	Acc@1	Acc@10	MRR@10	Index size	Search speed
256-dim	73.89%	97.45%	0.8188	1 × (baseline)	4 × nhanh hơn 1024
512-dim	75.16%	97.77%	0.8390	2 ×	2 × nhanh hơn 1024
1024-dim	78.34%	98.73%	0.8673	4 ×	baseline

MRL cho phép một checkpoint duy nhất hỗ trợ ba kích thước deployment. Đáng chú ý, 256-dim đạt đúng mức Acc@1 của baseline gốc (73.89%). Vì vậy, chọn 256-dim không làm mất chất lượng so với embedding ban đầu nhưng giúp index ChromaDB nhỏ hơn 4 lần và search nhanh hơn 4 lần.

Kết quả này mở ra chiến lược triển khai two-stage: search nhanh bằng 256-dim để lấy top-50 candidates, sau đó re-score bằng 1024-dim để chọn top-k cuối. Cấu

hình này phù hợp với môi trường production cần cân bằng giữa tốc độ, bộ nhớ và chất lượng.

5.3.3 So sánh với baseline

Table 5.10: So sánh embedding fine-tuned với baseline

Model	Acc@1	Acc@10	MRR@10	NDCG@10
Vietnamese_Embedding_v2 (baseline)	73.89%	98.09%	0.8259	0.7933
Fine-tuned (ours, 1024-dim)	78.34%	98.73%	0.8673	0.8538
Delta	+4.45%	+0.64%	+0.041	+0.061

Fine-tuned embedding cải thiện Acc@1 thêm 4.45 điểm phần trăm và MRR@10 thêm 0.041 so với Vietnamese_Embedding_v2 gốc trên cùng internal test gồm 157 queries. Cải thiện lớn nhất ở Acc@1, tức gold chunk xuất hiện ở rank 1, cho thấy model đã học được boundary ngữ nghĩa domain-specific tốt hơn, đặc biệt với câu hỏi về thông số kỹ thuật và bảng số liệu.

Acc@10 gần như giữ nguyên (98.09% $\rightarrow$ 98.73%) vì baseline đã đạt mức rất cao. Cải thiện chủ yếu nằm ở thứ tự ranking trong top-10, không phải recall tổng thể. Với retriever tốt hơn, nhiều query có gold chunk ở thứ hạng đầu hơn, mở rộng khả năng xử lý cho bước reranking tiếp theo.

5.4 Đánh giá Intent Classifier

Phần đánh giá PhoBERT intent classifier sẽ được hoàn thiện sau khi có kết quả thực nghiệm cuối cùng. Bảng 5.11 giữ lại các chỉ số cần báo cáo để đảm bảo quy trình đánh giá nhất quán.

Table 5.11: Khung kết quả đánh giá PhoBERT Intent Classifier

Metric	PhoBERT fine-tuned	GPT-4o-mini (v1)
Accuracy	TBD	99.39% (985/991)
F1-macro	TBD	—
F1 lớp tra_cuu	TBD	—
F1 lớp tính_toan	TBD	—
Latency/query	khoảng 1–3ms (local)	khoảng 800ms (API)
Chi phí/1M queries	\$0	khoảng \$30 tiền input với 200 tokens/query

Mục tiêu thiết kế của PhoBERT classifier là đạt accuracy $\geq 97\%$, ngưỡng đủ để routing không ảnh hưởng đáng kể đến accuracy end-to-end. Ngưỡng này dựa trên quan sát từ v1 rằng 6 câu bị nhầm intent vẫn được trả lời đúng cuối cùng. Đồng thời, latency mục tiêu dưới 5ms giúp bước routing không tạo overhead đáng kể

trong pipeline.

5.5 So sánh end-to-end v1 và v2

Phần accuracy end-to-end sẽ được bổ sung sau khi pipeline v2 hoàn chỉnh được chạy trên toàn bộ 343 câu hỏi thuộc 49 PDF unseen. Hiện tại, các chỉ số tài nguyên đã đo được cho thấy mức cải thiện rõ ràng về khả năng triển khai trên CPU.

Table 5.12: Khung so sánh end-to-end giữa v1 và v2

Chỉ số	v1 (Router-Summary)	v2 (tối ưu)	Delta
Accuracy trên 343 câu test	TBD	TBD	TBD
Latency reranker/query	khoảng 147ms (BGE-M3)	khoảng 22ms (MiniLM-ADR)	nhanh hơn $6.7\times$
Latency intent/query	khoảng 800ms (GPT API)	khoảng 3ms (PhoBERT local)	nhanh hơn $> 200\times$
Chi phí intent/1M queries	khoảng \$30 tiền input	\$0	-100%
Kích thước reranker	568M params	33M params	nhỏ hơn $17\times$

Số liệu latency v1 được đo trực tiếp từ thực nghiệm. BGE-M3 đạt khoảng 147ms/query trên CPU; GPT-4o-mini cần khoảng 800ms, bao gồm network round-trip. Số liệu v2 được đo trên cùng môi trường CPU. Accuracy end-to-end chỉ được báo cáo sau khi pipeline v2 hoàn chỉnh được đánh giá trên tập 343 câu hỏi zero-shot.

5.6 Phân tích lỗi tổng thể

Tổng hợp phân tích từ các thành phần cho thấy ba nguồn gốc lỗi chính theo thứ tự tác động giảm dần.

Nguồn 1 – Retrieval ceiling.

Trong 31 miss cases của reranker, 4 trường hợp xảy ra do gold chunk không nằm trong top-20 candidates, tức lỗi xuất hiện trước khi reranker nhìn thấy dữ liệu. Cải thiện embedding thêm 4.45 điểm phần trăm $\text{Acc}@1$ giúp giảm nhóm lỗi này nhưng không loại bỏ hoàn toàn. Các trường hợp khó nhất gồm acronym match (UPS, Offset) và bảng số liệu liền kề. Những trường hợp này cần cải thiện chunking strategy hoặc query expansion.

Nguồn 2 – Near miss.

15/31 miss cases có gold ở rank 6–10. Tăng top-K từ 5 lên 6 hoặc 7 có thể khôi phục thêm một phần các query này, nhưng phải đánh đổi với context window lớn hơn và chi phí generation cao hơn. Đây là trade-off cần được quyết định theo yêu cầu triển khai thực tế.

Nguồn 3 – Domain-specific confusion.

Cross-domain noise và same-domain confusion phản ánh giới hạn của embedding và BM25 với tài liệu kỹ thuật đặc thù. Các hướng cải thiện tiếp theo gồm bổ sung dữ liệu domain-specific cho embedding fine-tune, đặc biệt ở y tế, điện và HVAC, hoặc áp dụng query expansion để tăng recall trước khi rerank.

Nhìn tổng thể, pipeline v2 đạt $\text{Recall}@5 = 0.9155$ trên tập reranker zero-shot và $\text{Acc}@1 = 78.34\%$ trên internal test của embedding. Cả hai component đều cải thiện đáng kể so với baseline. Các giới hạn còn lại tập trung ở retrieval ceiling, dữ liệu domain-specific và chunking strategy; kết quả end-to-end cuối cùng sẽ được hoàn thiện sau khi chạy toàn bộ pipeline v2 trên 343 câu hỏi thuộc 49 PDF unseen.

CHƯƠNG 6. KẾT LUẬN

6.1 Contributions

Tổng hợp lại các đóng góp chính của đề án và điểm nổi bật nhất của hệ thống đề xuất.

6.2 Limitations

Nêu các giới hạn còn tồn tại về dữ liệu, chất lượng, độ trễ hoặc khả năng mở rộng.

6.3 Future Works

Đề xuất các hướng cải tiến và mở rộng tiếp theo cho hệ thống.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thống của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

[1] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

[2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

[3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

[4] M. Poesio and B. Di Eugenio, “Discourse structure and anaphoric accessibility,” in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

[5] A. Knott, “A data-driven methodology for motivating a set of coherence relations,” Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

[6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available: [ft](http://www.w3.org/Protocols/rfc2616/rfc2616-1.html)

`p:/info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

[7] Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO HUY

- [1] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993.
- [2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.
- [3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.
- [4] M. Poesio and B. Di Eugenio, “Discourse structure and anaphoric accessibility,” in *ESSLLI workshop on information structure, discourse structure and discourse semantics, Copenhagen, Denmark*, 2001, pp. 129–143.
- [5] A. Knott, “A data-driven methodology for motivating a set of coherence relations,” Ph.D. dissertation, The University of Edinburgh, UK, 1996.
- [6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. Accessed: Sep. 30, 2010. [Online]. Available: <ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z>
- [7] Princeton University, *Wordnet*. Accessed: Sep. 30, 2010. [Online]. Available: <http://www.cogsci.princeton.edu/~wn/index.shtml>

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cầu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi

trong từng trang báo cáo ĐATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

- Đối với kỹ sư chính quy:
 - Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - Từ K62 trở về sau: Ngành Khoa học máy tính
- Đối với cử nhân:
 - Ngành Công nghệ thông tin
- Đối với chương trình EliteTech:
 - Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - Chương trình ICT Global: Ngành Information Technology
 - Chương trình DS&AI: Ngành Khoa học dữ liệu

A.2 Đánh dấu (bullet) và đánh số (numering)

Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```
- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
```

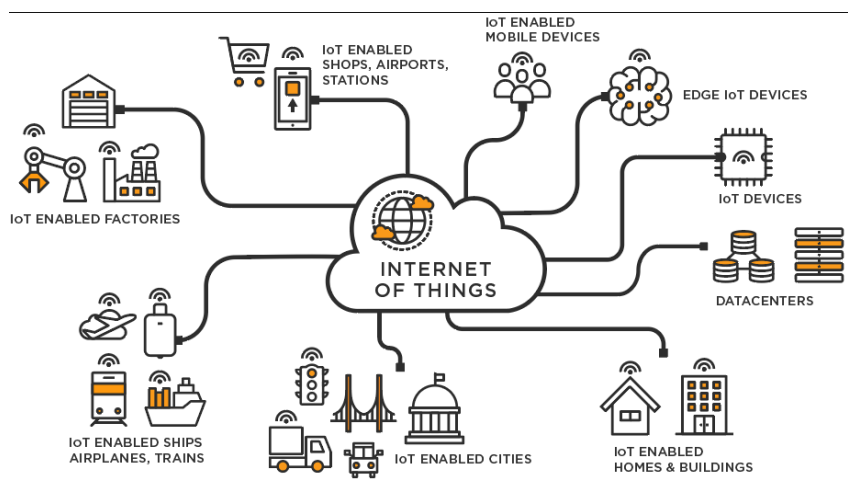


Figure A.1: Internet vạn vật

\end{enumerate}

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Table A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong Latex khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong Latex tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được

phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau [1]. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau [2]. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.2 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:

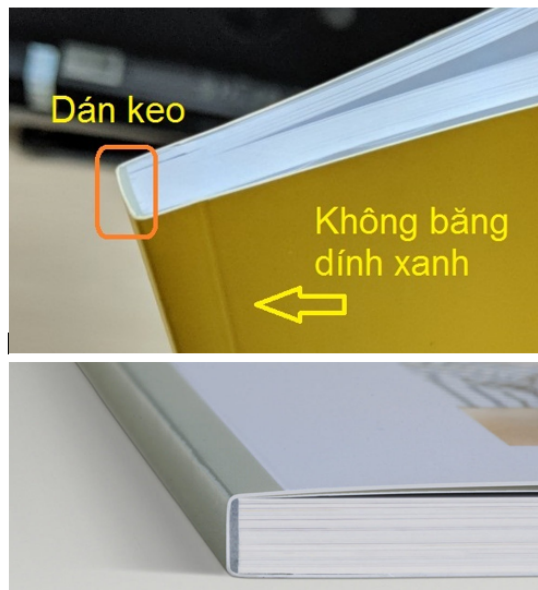


Figure A.2: Qui cách đóng quyển đồ án

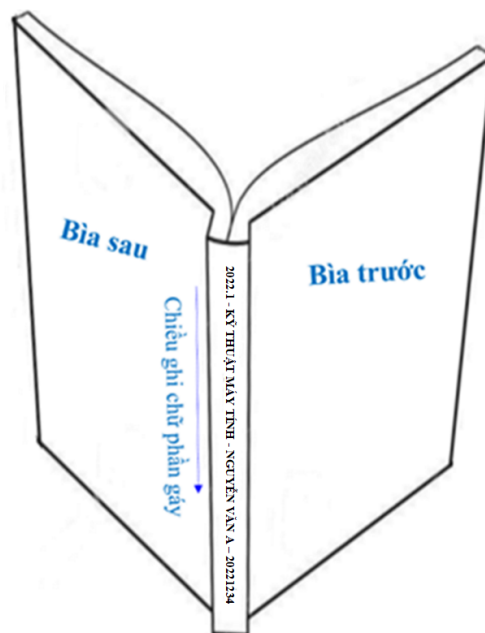


Figure A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Thống kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...